A visual field manual

Swift for the Coding Interview

Syntax, data structures, the patterns behind the problems, and the iOS fundamentals the rest of the loop is built on.

The language · Data structures · Patterns · The iOS engineer · Reference

How to read this book

Three parts, meant to be read in order once and then raided out of order forever.

Part one is the Swift a coding interview actually exercises. Not the whole language — no property wrappers, no actors, no SwiftUI. Just the parts you will type under time pressure, plus the handful of places where Swift behaves differently from the languages most interview material is written in.

Part two is data structures, each one with the same three questions answered: what shape is it, what does it cost, and how do you write it in Swift when the standard library doesn't give it to you. Swift ships without a heap, a queue, a linked list, or a trie. You will write all four.

Part three is the patterns. This is the part that changes your hit rate. Most interview problems are a small number of shapes wearing different clothes, and the skill being tested is recognising the shape in the first two minutes.

Part four is the rest of the loop. An algorithm round is one interview out of four or five; the others ask about memory, concurrency, the frameworks, and how you would build a real screen. These chapters cover what an iOS engineer is expected to know cold, and they are the ones that separate a strong Swift programmer from a strong iOS candidate.

> **How to actually use it**
>
> Read a pattern chapter, close the book, and write the template from memory before you look at any problem. If you cannot reproduce the template cold, you do not know the pattern yet — you have only recognised it. Recognition disappears under interview stress; reproduction does not.

> **A note on the code**
>
> Every snippet is written the way you would want to write it in a real interview: named clearly, no cleverness for its own sake, and short enough to hold in your head. Where a shorter or more "Swifty" version exists but would be harder to explain out loud, the explainable version wins.

Contents

Part one

The language you actually need

Swift is a big language. A coding interview uses maybe a tenth of it. This part covers that tenth, and lingers on the four or five places where Swift will surprise you.

Chapter one

Values, types and the type system

Swift decides the type of everything at compile time, and refuses to guess. That strictness costs you a few keystrokes and saves you the entire category of bugs where a number quietly became a string.

Constants and variables

`let` makes a binding you cannot reassign. `var` makes one you can. Reach for `let` first and let the compiler tell you when you were wrong — it is a small habit that interviewers read as fluency.

Declaring things

```
let limit = 100          // Int, inferred, cannot change
var count = 0            // Int, inferred, can change
count += 1

let name: String = "Ada" // annotated explicitly
var ratio: Double = 0.5

let sum = limit + count  // both Int, fine
```

Type inference works from the right-hand side. When there is nothing to infer from, you must annotate:

```
var seen: Set<Int> = []          // [] alone is ambiguous
var counts: [String: Int] = [:]  // so is [:]
var path = [Int]()               // the other spelling, equally common
```

Swift will not convert numbers for you

There is no implicit widening. An `Int` and a `Double` cannot be added, and an `Int` and an `Int32` cannot either. You convert explicitly by constructing the type you want.

```
let n = 7
let half = Double(n) / 2.0   // 3.5   — convert first
let bad  = n / 2             // 1     — integer division truncates
let idx  = Int(3.99)         // 3     — Int(_:) truncates toward zero
```

Integer division truncating toward zero is the single most common source of off-by-one answers in interview code. When you write `(low + high) / 2`, you are relying on it deliberately. Everywhere else, check that you meant it.

The types you will use

Type	What it holds	Notes for interviews
`Int`	whole number, 64-bit on modern devices	the default; `Int.max`, `Int.min`
`Double`	floating point	avoid for exact comparisons
`Bool`	`true` / `false`	no truthiness — `if 1` does not compile

Type	What it holds	Notes for interviews
`String`	text	see chapter 4, it is not an array
`Character`	one grapheme	what you get iterating a `String`
`[T]`	array of `T`	`Array<T>` in longhand
`[K: V]`	dictionary	`Dictionary<K, V>` in longhand
`Set<T>`	unordered unique	`T` must be `Hashable`
`(A, B)`	tuple	great for returning two things

Overflow is a crash, not a wraparound

```
let big = Int.max
// let boom = big + 1     // runtime trap: arithmetic overflow
let wrapped = big &+ 1    // Int.min — the &+ operator wraps on purpose
```

This matters when a problem says "the answer fits in a 32-bit integer". If you are asked to detect overflow, the ampersand operators and the `addingReportingOverflow` family are the tools:

```
let (value, didOverflow) = Int32(2_000_000_000).addingReportingOverflow(2_000_000_000)
// value = -294967296, didOverflow = true
```

Tuples

A tuple is an ad-hoc bundle. It is the cheapest way to return two values without inventing a type, and it destructures on the way out.

Returning two things

```
func minMax(_ nums: [Int]) -> (low: Int, high: Int)? {
    guard let first = nums.first else { return nil }
    var low = first, high = first
    for n in nums {
        low = min(low, n)
        high = max(high, n)
    }
    return (low, high)
}

if let result = minMax([3, 1, 4, 1, 5]) {
    print(result.low, result.high)   // 1 5
}

let (a, b) = (1, 2)   // destructuring
```

Tuples also compare lexicographically, which is occasionally exactly what you want when sorting by two keys:

```
let people = [("Bea", 30), ("Ada", 30), ("Cy", 25)]
let sorted = people.sorted { ($0.1, $0.0) < ($1.1, $1.0) }
// sorts by age, then by name
```

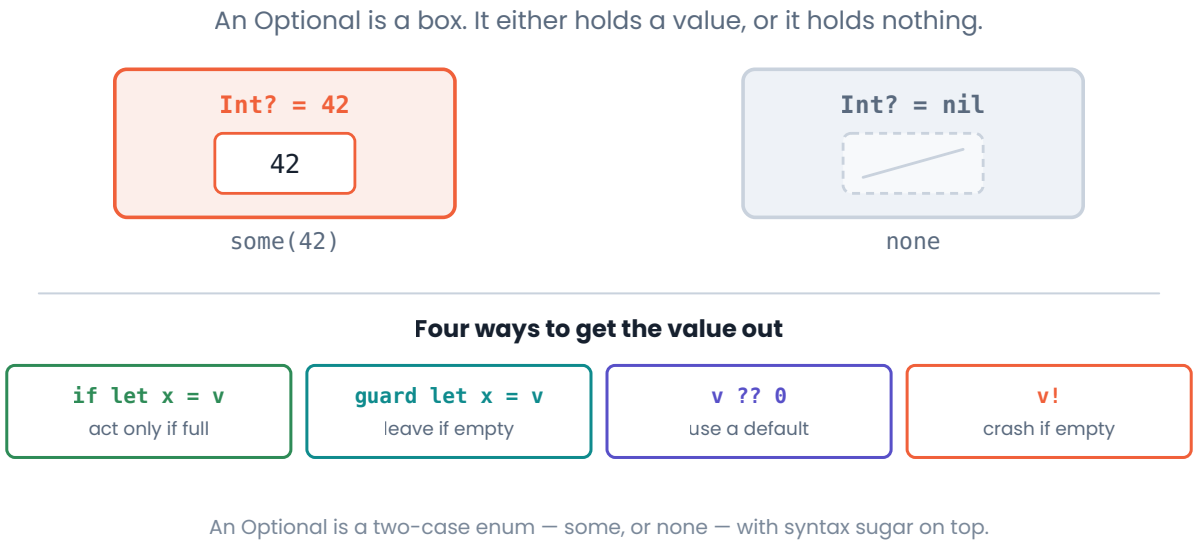
Say this out loud

"I'll use `let` unless I need to mutate it" and "I'll convert explicitly here because Swift won't widen the `Int`" are two sentences that tell an interviewer you write Swift rather than translate from another language into it.

Chapter two

Optionals

An Optional is Swift's answer to null. The difference is that the type system knows, so the compiler forces you to deal with absence at the moment it can happen rather than three stack frames later.



An Optional is a two-case enum — some, or none — with syntax sugar on top.

`Int?` is shorthand for `Optional<Int>`, an enum with exactly two cases. You cannot use the value inside until you take it out, and Swift gives you four ways to do that.

if let — act only when there is something

```
let raw: String = "42"
if let number = Int(raw) {
    print(number * 2)      // 84, and `number` is a plain Int in here
} else {
    print("not a number")
}

// Swift 5.7 and later, when the names match:
if let number { print(number) }
```

guard let — leave early when there is nothing

`guard` is the one to prefer at the top of a function. It keeps the happy path unindented, which matters a lot when someone is reading your code on a screen share.

```
func firstWord(of sentence: String) -> String? {
    guard let word = sentence.split(separator: " ").first else {
        return nil
    }
    return String(word)      // `word` stays in scope below the guard
}
```

The `else` block of a `guard` must leave the current scope: `return`, `throw`, `break`, `continue`, or `fatalError()`. That is the rule the compiler enforces, and it is why `guard` bindings survive afterwards while `if let` bindings do not.

?? — supply a default

```
let counts: [String: Int] = ["a": 3]
let n = counts["b"] ?? 0     // 0, because "b" is missing
```

Optional chaining — reach through, get nothing back if anything was nothing

```
let firstLength = ["hi", "there"].first?.count   // Optional(2)
let missing: [String] = []
let none = missing.first?.count              // nil, no crash
```

And the one to avoid

```
let value = Int("oops")!    // crash: unexpectedly found nil
```

Force unwrapping is not banned, but every `!` in an interview is a question you have invited. Use it only where you can say in one sentence why nil is impossible.

Where Optionals appear when you did not ask for them

- `dict[key]` is always Optional — the key may not be there.
- `array.first` and `array.last` are Optional — the array may be empty.
- `Int("abc")` is Optional — the text may not be a number.
- `array.firstIndex(of:)` is Optional — it may not be found.

Note the asymmetry: `dict[key]` is Optional but `array[index]` is not. An out-of-range array index crashes instead. Swift assumes you checked.

Optionals in the middle of an algorithm

The pattern you will use most often is walking a linked list, where `next` is Optional by necessity — the last node has to point at something, and that something is nothing:

```
var current: ListNode? = head
while let node = current {      // loop while there is a node
    print(node.value)
    current = node.next         // eventually nil, and the loop ends
}
```

`while let` is the same idea as `if let`, repeated. It reads almost as prose, and it is the idiomatic Swift way to walk any Optional chain.

The question behind the question

If an interviewer asks "what is an Optional", they are rarely asking for a definition. They want to hear that it is a compile-time guarantee rather than a runtime check — that the reason it exists is to make the absent case impossible to forget, and that force-unwrapping throws away the guarantee.

Chapter three

Arrays, dictionaries and sets

Three collections carry almost every interview solution. Learn their literal syntax cold, because fumbling `[:]` at the whiteboard costs you thirty seconds and some composure.

Array

Creating and reading

```
var nums = [3, 1, 4, 1, 5]
let zeros = Array(repeating: 0, count: 5)      // [0, 0, 0, 0, 0]
let grid  = Array(repeating: Array(repeating: 0, count: 3), count: 2)

nums[0]           // 3        — not Optional; out of range crashes
nums.first        // Optional(3)
nums.count        // 5
nums.isEmpty      // false
```

Changing

```
nums.append(9)                // O(1) amortized
nums += [2, 6]                // append a whole sequence
nums.insert(0, at: 0)         // O(n) — everything shifts right
nums.removeLast()             // O(1), returns the element
nums.removeFirst()            // O(n) — everything shifts left
nums.remove(at: 2)            // O(n)
nums.reverse()                // in place; `reversed()` returns a view
nums.sort()                   // in place; `sorted()` returns a new array
```

Iterating

```
for n in nums { }                          // values
for (i, n) in nums.enumerated() { }        // index and value
for i in 0..<nums.count { }                // index only
for i in stride(from: nums.count - 1, through: 0, by: -1) { }   // backwards
for (a, b) in zip(nums, nums.dropFirst()) { }                   // pairs
```

The two-dimensional array trap

Write `Array(repeating: Array(repeating: 0, count: cols), count: rows)` and you get genuinely independent rows, because arrays are value types and each row is copied. The same line in Python would give you `rows` references to one list. This is one of the rare cases where Swift's value semantics quietly save you.

Dictionary

```
var counts: [String: Int] = [:]
counts["a"] = 1
counts["a"]                    // Optional(1)
counts["z"]                    // nil
counts["z", default: 0] += 1   // the counting idiom — creates it at 0 first
counts.removeValue(forKey: "a")

for (key, value) in counts { }     // order is not defined
counts.keys                        // a collection view
counts.values
counts.count
```

The `default:` subscript is worth memorising. Frequency counting appears in an enormous share of string and array problems, and this is the one-liner:

Frequency map in one line

```
let freq = text.reduce(into: [Character: Int]()) { $0[$1, default: 0] += 1 }
```

Set

```
var seen: Set<Int> = []
seen.insert(3)                 // returns (inserted: Bool, memberAfterInsert: Int)
seen.contains(3)               // O(1) average
seen.remove(3)

let a: Set = [1, 2, 3], b: Set = [3, 4]
a.union(b)          // {1, 2, 3, 4}
a.intersection(b)   // {3}
a.subtracting(b)    // {1, 2}
a.isSubset(of: b)   // false
```

A `Set` is the right answer any time the question is "have I seen this before". It is also the fastest way to deduplicate: `Array(Set(nums))`, at the cost of losing the order.

Transforming — the four you will actually use

```
let nums = [1, 2, 3, 4, 5]

nums.map { $0 * $0 }               // [1, 4, 9, 16, 25]
nums.filter { $0 % 2 == 0 }        // [2, 4]
nums.reduce(0, +)                  // 15
nums.reduce(0) { $0 + $1 * 2 }     // 30

["1", "x", "3"].compactMap { Int($0) }     // [1, 3] — drops the nils
[[1, 2], [3]].flatMap { $0 }               // [1, 2, 3] — one level flatter

nums.sorted { $0 > $1 }            // descending
nums.contains(3)                   // O(n) for an array
nums.allSatisfy { $0 > 0 }         // true
nums.min(); nums.max()             // both Optional
```

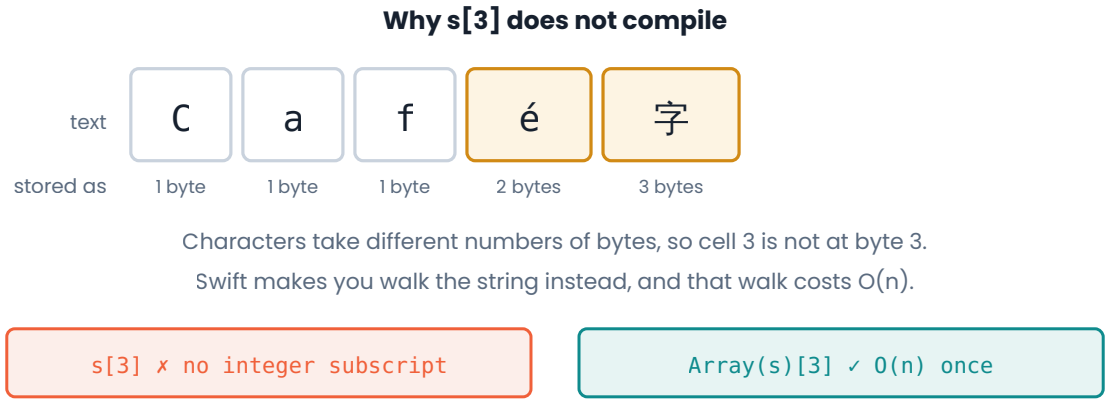
When not to be clever

A chain of `map`, `filter` and `reduce` reads beautifully and is often the right answer. But if you need an early exit, or you are mutating two things at once, a plain `for` loop is clearer and no slower. Interviewers care that you can explain the code, not that it fit on one line.

Chapter four

Strings, and why they are strange

Swift's `String` is the one type that behaves differently from what interview problems assume. Understand why once, and the workaround becomes automatic.



A Character is a grapheme cluster, and clusters have different byte lengths.

The problem

A Swift `String` is a sequence of `Character` values, and a `Character` is a grapheme cluster — what a human would call a single letter, even when it is built from several Unicode scalars. Because clusters have different byte widths, there is no arithmetic that jumps to "the fourth character". You have to walk.

Swift makes that cost visible by refusing to give you `s[3]` at all.

```
let s = "hello"
// s[3]                          // does not compile
let i = s.index(s.startIndex, offsetBy: 3)
s[i]                             // "l" — O(n) to find the index
```

The fix

For any problem that indexes into a string more than once, convert to an array of characters up front. You pay O(n) once and then index freely.

The move you will make in nearly every string problem

```
let chars = Array(s)     // [Character], O(n)
chars[3]                 // "l", O(1)
chars.count              // O(1)

// ...and back again when you are done
let result = String(chars)
```

Why this is not a hack

Converting once and indexing many times is genuinely the right complexity trade. What would be a mistake is calling `s.index(s.startIndex, offsetBy: i)` inside a loop — that turns an O(n) algorithm into O(n²) without changing a single line of your logic. Say this out loud when you convert; interviewers notice.

The operations you need

```
var s = "Hello, world"

s.count                          // 12 — O(n), it counts
s.isEmpty
s.uppercased(); s.lowercased()
s.hasPrefix("Hello"); s.hasSuffix("d")
s.contains("wor")
s.append("!"); s += "!"
String(s.reversed())             // "!!dlrow ,olleH"

s.split(separator: ",")          // [Substring] — note the type
s.split(separator: ",").map(String.init)
s.trimmingCharacters(in: .whitespaces)
s.replacingOccurrences(of: "l", with: "L")
```

`split` gives you `Substring`, not `String`. A `Substring` shares storage with its parent, which is efficient but keeps the whole original string alive. Convert with `String(...)` when you store it; leave it alone when you are just looking.

Characters and ASCII arithmetic

Counting letters is common enough that the conversion to a number is worth knowing:

Character to index

```
let c: Character = "c"
let offset = Int(c.asciiValue! - Character("a").asciiValue!)   // 2

// a 26-slot counter, the classic anagram check
func isAnagram(_ a: String, _ b: String) -> Bool {
    guard a.count == b.count else { return false }
    var counts = Array(repeating: 0, count: 26)
    let base = Character("a").asciiValue!
    for c in a { counts[Int(c.asciiValue! - base)] += 1 }
    for c in b {
        let i = Int(c.asciiValue! - base)
        counts[i] -= 1
        if counts[i] < 0 { return false }
    }
    return true
}
```

`asciiValue` is Optional because not every `Character` is ASCII. Force-unwrapping is defensible here only if the problem promises lowercase English letters — and if it does, say so before you type the `!`.

Useful character tests, all on `Character`:

```
c.isLetter; c.isNumber; c.isUppercase; c.isLowercase
c.isWhitespace; c.isPunctuation
c.wholeNumberValue       // Optional<Int>: "7" -> 7
```

A worked example: valid palindrome

Ignoring case and punctuation

```swift
func isPalindrome(_ s: String) -> Bool {
    let chars = Array(s.lowercased().filter { $0.isLetter || $0.isNumber })
    var left = 0, right = chars.count - 1
    while left < right {
        if chars[left] != chars[right] { return false }
        left += 1
        right -= 1
    }
    return true
}
```

Cost O(n) time — one filter pass plus one two-pointer pass. O(n) space for the filtered array. Doing it without the extra array means skipping non-alphanumerics inside the while loop, which is O(1) space and a good follow-up to offer.

Chapter five

Control flow and pattern matching

Loops are unremarkable. `switch` is not — Swift's version matches on shapes, ranges and conditions, and the compiler checks you covered everything.

Loops and ranges

```
for i in 0..<5 { }        // 0,1,2,3,4   — half-open
for i in 0...5 { }        // 0,1,2,3,4,5 — closed
for i in (0..<5).reversed() { }
for i in stride(from: 0, to: 10, by: 2) { }       // 0,2,4,6,8
for i in stride(from: 10, through: 0, by: -2) { } // 10,8,6,4,2,0

while condition { }
repeat { } while condition        // Swift's do-while
```

Ranges must go forwards

`for i in 5..<0` is a runtime crash, not an empty loop. If a bound is computed and could invert, guard it or use `stride`, which handles direction properly.

switch, which is doing more than you think

Every `switch` must be exhaustive. Cases do not fall through. There is no `break` needed.

```
switch score {
case ..<0:      print("invalid")
case 0:         print("zero")
case 1..<50:    print("low")
case 50...79:   print("mid")
default:        print("high")
}
```

It matches tuples, which is how you compare two things at once without nesting `if`s:

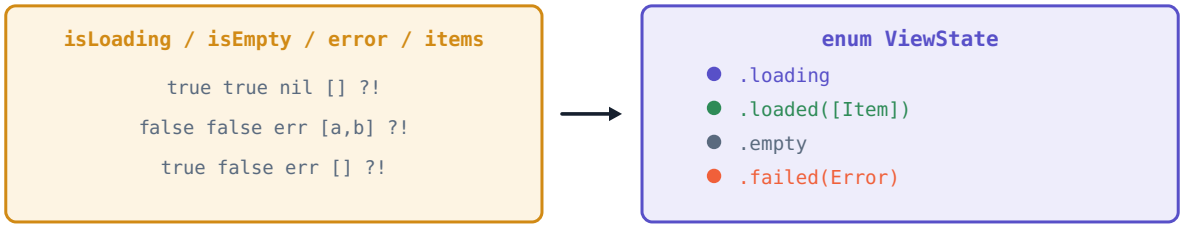
```
switch (row, col) {
case (0, 0):            print("origin")
case (_, 0):            print("left edge")
case (let r, let c) where r == c: print("diagonal")
default:                break
}
```

And it binds associated values out of enums, which is where it earns its keep:

A load state, the shape you will write again and again

```
enum LoadState {
    case loading
    case loaded([String])
    case empty
    case failed(Error)
}

switch state {
case .loading:            showSpinner()
case .loaded(let items):  show(items)
case .empty:              showEmptyMessage()
case .failed(let error):  show(error.localizedDescription)
}
```

One enum replaces four fragile booleans



16 combinations. 12 of them are nonsense.

Exactly 4 states. All of them are real.

The compiler now forces you to handle every case in the switch.

Exhaustiveness is the point — add a case, and every switch stops compiling until you handle it.

Why this beats booleans

Four booleans describe sixteen states, twelve of which are contradictions you now have to defend against at runtime. One enum with four cases describes four states, and the compiler will not let you forget one. This is the single most useful design idea Swift gives you, and it applies as much to algorithm code as to UI code.

guard, defer and labelled breaks

```
func process(_ input: [Int]?) -> Int {
    guard let input, !input.isEmpty else { return 0 }
    return input.reduce(0, +)
}
```

Labelled breaks matter in grid problems, where you want to leave both loops at once:

```
outer: for row in grid {
    for cell in row {
        if cell == target { found = true; break outer }
    }
}
```

`continue` skips to the next iteration; `break` leaves the innermost loop unless you label it.

Chapter six

Functions and closures

Swift function signatures carry argument labels, which makes call sites read like sentences and makes the declaration look busier than you expect at first.

Labels, defaults, and the underscore

```
func move(from start: Int, to end: Int) -> Int { end - start }
move(from: 2, to: 9)          // labels are required at the call site

func twoSum(_ nums: [Int], _ target: Int) -> [Int] { [] }
twoSum([1, 2], 3)             // `_` means "no label" — standard for algorithms

func greet(_ name: String, punctuation: String = "!") -> String {
    "Hello, \(name)\(punctuation)"
}
greet("Ada")                  // defaults fill in
```

A single-expression function can drop the `return`, which is why `{ end - start }` above compiles.

Parameters are constants

You cannot reassign a parameter. Copy it, or mark it `inout` to write through to the caller's variable.

```
func normalise(_ text: String) -> String {
    var working = text        // parameters are `let`; make your own copy
    working = working.lowercased()
    return working
}

func bump(_ value: inout Int) { value += 1 }
var count = 0
bump(&count)                  // the & is required and visible at the call site
```

Closures

A closure is a function without a name. The full form is verbose; the shortened forms are what you will actually write.

The same closure, four ways

```
let byLength = names.sorted(by: { (a: String, b: String) -> Bool in
    return a.count < b.count
})
let b2 = names.sorted(by: { a, b in a.count < b.count })   // types inferred
let b3 = names.sorted { a, b in a.count < b.count }        // trailing closure
let b4 = names.sorted { $0.count < $1.count }              // shorthand arguments
```

`$0` and `$1` are the first and second arguments. They are idiomatic for short closures and unreadable for long ones — if the body is more than a line or two, name the parameters.

Closures capture, and capture is by reference

```
var total = 0
let add = { (n: Int) in total += n }   // captures `total` itself, not a copy
add(5)
print(total)   // 5
```

This is what makes a recursive helper closure work, and it is also how you accidentally keep an object alive in real iOS code. In interviews the relevant fact is the first one: an inner function can read and write the outer function's variables, which is exactly what you want for DFS.

The nested-function pattern you will use constantly

```
func subsets(_ nums: [Int]) -> [[Int]] {
    var result: [[Int]] = []
    var path: [Int] = []

    func backtrack(_ start: Int) {          // sees result and path directly
        result.append(path)
        for i in start..<nums.count {
            path.append(nums[i])
            backtrack(i + 1)
            path.removeLast()
        }
    }

    backtrack(0)
    return result
}
```

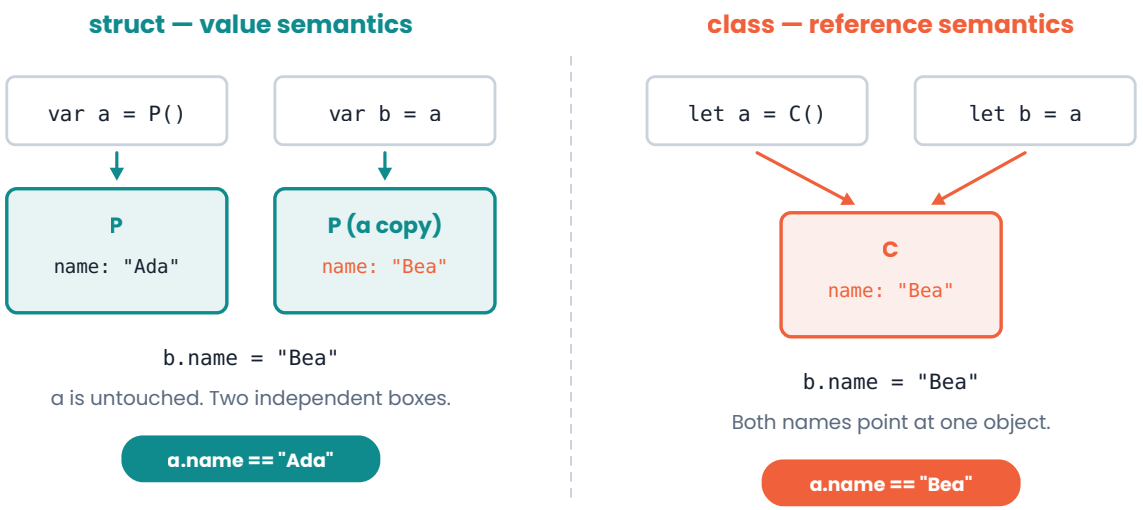
Prefer a nested function to a closure for recursion

A closure that calls itself needs to be declared before it can reference itself, which forces an awkward two-step. A nested `func` can recurse immediately, captures the enclosing variables the same way, and reads better. This is the standard shape for DFS and backtracking in Swift.

Chapter seven

Structs, classes and enums

One decision explains most of the difference: a struct is copied, a class is shared. Everything else follows from that.



Assignment copies a struct and shares a class. This is the whole distinction.

Struct

```
struct Point {
    var x: Int
    var y: Int                  // memberwise init is generated for you

    func distance(to other: Point) -> Double {
        let dx = Double(x - other.x), dy = Double(y - other.y)
        return (dx * dx + dy * dy).squareRoot()
    }

    mutating func shift(by d: Int) {   // `mutating` required to change self
        x += d
        y += d
    }
}

var p = Point(x: 1, y: 2)
p.shift(by: 3)                  // only works because `p` is a var
```

A method that changes a struct's own properties must be marked `mutating`, and you can only call it on a `var`. That is the type system reminding you that mutating a value means replacing it.

Class

```
class Node {
    var value: Int
    var next: Node?             // Optional, because the last node has no next

    init(_ value: Int) {        // classes need an explicit init
        self.value = value
    }
}
```

Classes get reference semantics, inheritance, and identity comparison with `===`. For interview code, the practical rule is short:

> Use a **struct** for data you pass around and compare. Use a **class** for nodes that point at each other — linked lists and trees — because a value type cannot contain itself.

That last point is not a style preference. `struct Node { var next: Node? }` will not compile, because the compiler cannot compute a finite size for it. Linked structures need reference types, or an explicit `indirect enum`.

Enum

Enums in Swift are far more than named integers. They can carry values, have methods, and be recursive.

```
enum Direction {
    case north, south, east, west

    var opposite: Direction {
        switch self {
        case .north: return .south
        case .south: return .north
        case .east:  return .west
        case .west:  return .east
        }
    }
}

enum Token {
    case number(Int)           // associated values: each case can carry data
    case operatorSymbol(Character)
    case end
}
```

Raw values give you a two-way mapping to a primitive, and `CaseIterable` gives you the list:

```
enum Suit: String, CaseIterable {
    case hearts, spades, clubs, diamonds
}
Suit.allCases.count            // 4
Suit(rawValue: "spades")       // Optional(.spades)
Suit.hearts.rawValue           // "hearts"
```

Equatable, Hashable, Comparable

To put your type in a `Set`, use it as a dictionary key, or sort it, it must conform to the right protocol. For simple types Swift synthesises all of them if you just ask:

```
struct Point: Hashable {       // Equatable comes free with Hashable
    var x: Int
    var y: Int
}

var visited: Set<Point> = []   // now legal
visited.insert(Point(x: 0, y: 0))
```

This is the clean way to track visited cells in a grid problem when you do not want a second 2-D array. The synthesis works as long as every stored property is itself `Hashable`.

Chapter eight

Errors, generics and protocols

The last of the language. Interviews rarely test these directly, but reaching for them at the right moment is what separates Swift code from translated Java.

Throwing and catching

```
enum ParseError: Error {
    case empty
    case notANumber(String)
}

func parse(_ text: String) throws -> Int {
    guard !text.isEmpty else { throw ParseError.empty }
    guard let n = Int(text) else { throw ParseError.notANumber(text) }
    return n
}

do {
    let n = try parse("42")
    print(n)
} catch ParseError.notANumber(let text) {
    print("bad input: \(text)")
} catch {
    print("something else: \(error)")
}
```

Two shorthands are worth knowing. `try?` turns a throw into `nil`, and `try!` turns it into a crash:

```
let maybe = try? parse("x")    // nil, of type Int? — try? does not double-wrap
let sure  = try! parse("42")   // traps if it throws
```

Generics

You write a generic function once and it works for every type that satisfies the constraint. The constraint is the interesting part.

```
func firstDuplicate<T: Hashable>(in items: [T]) -> T? {
    var seen: Set<T> = []
    for item in items {
        if seen.contains(item) { return item }
        seen.insert(item)
    }
    return nil
}

firstDuplicate(in: [1, 2, 1])          // Optional(1)
firstDuplicate(in: ["a", "b", "a"])    // Optional("a")
```

`<T: Hashable>` reads as "for any type T that can be hashed". Without the constraint, `Set<T>` would not compile — Swift needs to know T can be put in a set. The common constraints in algorithm code are `Hashable`, `Equatable` and `Comparable`.

Protocols

A protocol is a list of requirements. A type that conforms promises to meet them.

```
protocol Container {
    associatedtype Item
    var count: Int { get }
    mutating func add(_ item: Item)
}

struct Stack<T>: Container {
    private var storage: [T] = []
    var count: Int { storage.count }
    mutating func add(_ item: T) { storage.append(item) }
}
```

For interview purposes the ones that matter are the standard library's: `Hashable` to be a dictionary key or set member, `Comparable` to use `<` and `sorted()`, `Equatable` for `==`, and `CustomStringConvertible` to control `print`.

The design question at the end

Many iOS interviews finish an algorithm round with "how would you structure this in an app". Protocols and generics are the vocabulary for that answer: define behaviour as a protocol so the caller can be tested against a fake, keep the concrete type behind it, and prefer composition to inheritance. You do not need depth here — you need to sound like someone who has made the choice before.

Part two

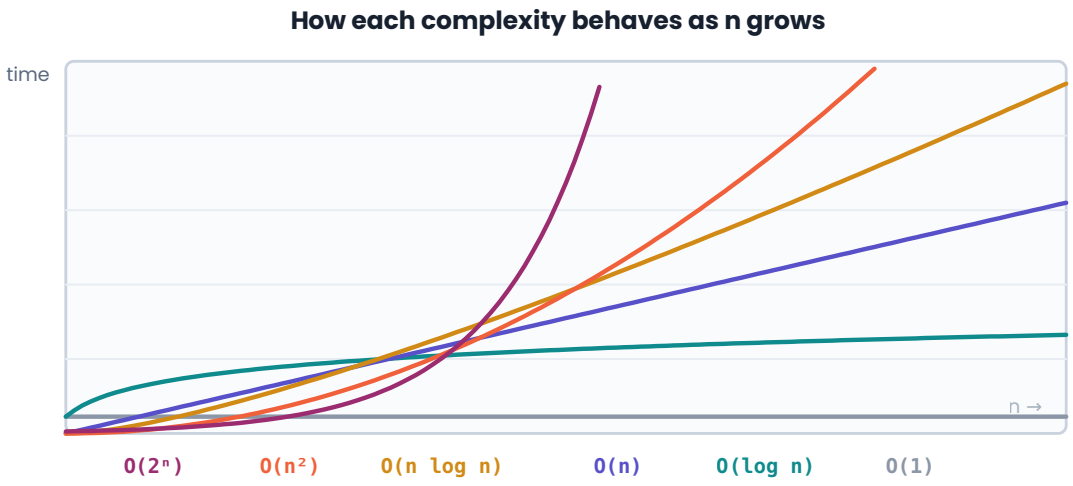
Data structures, and what they cost

Swift's standard library gives you three collections and stops. The rest — stacks, queues, linked lists, heaps, graphs, tries — you build. That is not a gap; it is the interview.

Chapter nine

Reading complexity

Big-O is not a grade. It is a claim about what happens to the running time when the input gets much bigger, and it is the language you will use to justify every decision you make out loud.



Anything steeper than the indigo line will time out on large inputs.

The gaps only open up as n grows — which is exactly the regime interviewers ask about.

What the notation is actually saying

O(n) means: as n grows, the work grows at most in proportion to n. Constants and lower-order terms are dropped, because they stop mattering. An algorithm that does `3n + 50` steps is O(n), and so is one that does `n / 2`.

That is a deliberate coarseness. It lets you compare approaches before writing either of them, which is the whole point during the first five minutes of an interview.

Growth	Name	What causes it
O(1)	constant	a hash lookup, arithmetic, array indexing
O(log n)	logarithmic	halving the search space each step
O(n)	linear	one pass over the input
O(n log n)	linearithmic	sorting, or a linear pass doing a log operation
O(n²)	quadratic	nested loops over the same input
O(2ⁿ)	exponential	trying every subset
O(n!)	factorial	trying every ordering

Reading it off your own code

Nested loops multiply. Sequential loops add, and then the smaller one is dropped. A recursive call multiplies by the number of branches, raised to the depth.

```swift
for a in nums {              // n
    for b in nums {          // × n
        if a + b == target { }
    }
}                            // O(n²)

nums.sort()                  // O(n log n)
for n in nums { }            // + O(n)  →  dropped
                             // O(n log n) overall
```

The costs that hide inside a one-liner

Swift makes several O(n) operations look like O(1). `array.contains(x)` scans. `array.removeFirst()` shifts. `string.count` counts. `array.insert(_:at:)` shifts. Putting any of them inside a loop silently turns O(n) into O(n²), and this is the most common way an otherwise correct interview solution fails the follow-up.

Space complexity counts too

Space is the extra memory you allocate, not counting the input. A recursive function's call stack counts: depth-n recursion is O(n) space even if you allocate nothing.

```swift
func sum(_ nums: [Int], _ i: Int = 0) -> Int {
    i == nums.count ? 0 : nums[i] + sum(nums, i + 1)   // O(n) stack
}
```

Say the complexity before you are asked

Finish every solution with one sentence: "this is O(n) time and O(n) space, because I make one pass and store at most one entry per element". Volunteering it turns a question you might be caught by into a demonstration of fluency. If you can also name what you would trade to improve it, you are done.

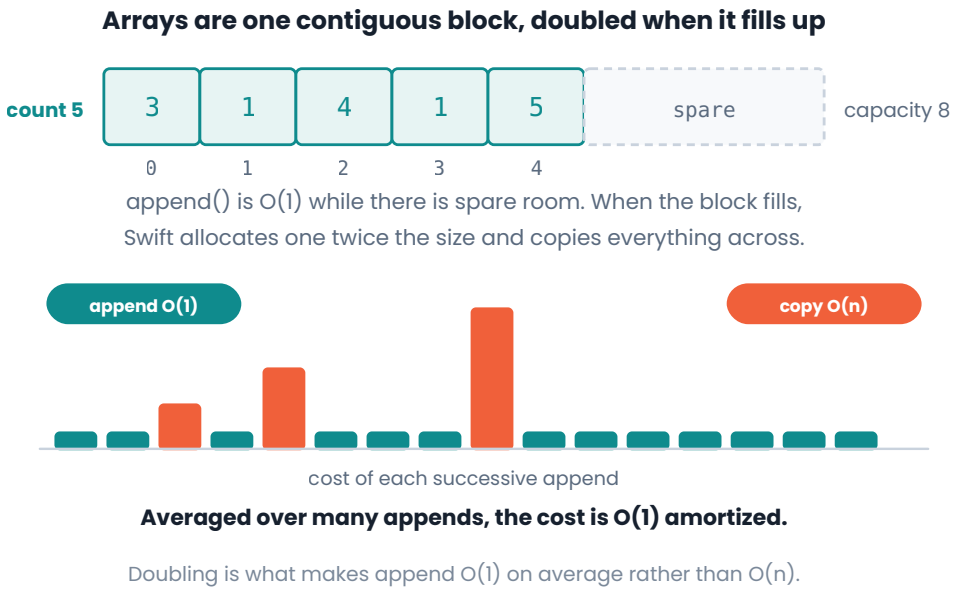
What "n" means when there are two inputs

Be precise. If you merge two arrays of length m and n, the answer is O(m + n), not O(n). If you compare every word in a list of n words with average length k, it is O(n·k), and interviewers do listen for the k.

Chapter ten

Arrays

One contiguous block of memory with a count and a capacity. Everything an array is fast or slow at follows from that sentence.



Costs

Operation	Cost	Why
`nums[i]`	O(1)	address arithmetic
`append`	O(1) amortized	occasional doubling, spread out
`insert(at:)`	O(n)	shift everything right
`remove(at:)`	O(n)	shift everything left
`removeLast`	O(1)	nothing to shift
`removeFirst`	O(n)	shift everything left
`contains`	O(n)	linear scan
`sort`	O(n log n)	a modified timsort
`min` / `max`	O(n)	one pass

Reserving capacity

If you know the final size, say so. It removes the reallocations entirely, and mentioning it is a cheap signal that you think about allocation.

```
var result: [Int] = []
result.reserveCapacity(nums.count)
```

Value semantics and copy-on-write

An array is a struct, so assigning it copies it. Swift makes this affordable with copy-on-write: the copy shares the same buffer until one side writes, at which point the write triggers the real copy.

```
var a = [1, 2, 3]
var b = a          // no copy yet — both point at one buffer
b.append(4)        // now the buffer is copied, and a is untouched
print(a)           // [1, 2, 3]
```

> **Why this matters in a recursive function**
>
> Passing an array into each recursive call looks free and is not: the moment the callee mutates it, you pay a full O(n) copy at every level. This is why backtracking solutions keep **one** shared path array and append and remove from it, rather than passing modified copies down.

Two-dimensional arrays

```
var grid = Array(repeating: Array(repeating: 0, count: cols), count: rows)
grid[r][c] = 1

for r in 0..<rows {
    for c in 0..<cols { }
}

// the four neighbours, the way you will write it every time
let directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
for (dr, dc) in directions {
    let nr = r + dr, nc = c + dc
    guard nr >= 0, nr < rows, nc >= 0, nc < cols else { continue }
    // visit grid[nr][nc]
}
```

That `guard` with four conditions is worth committing to muscle memory. Bounds checking is where grid problems actually go wrong, and writing it once at the top of the loop beats scattering `if`s through the body.

Sorting with a custom order

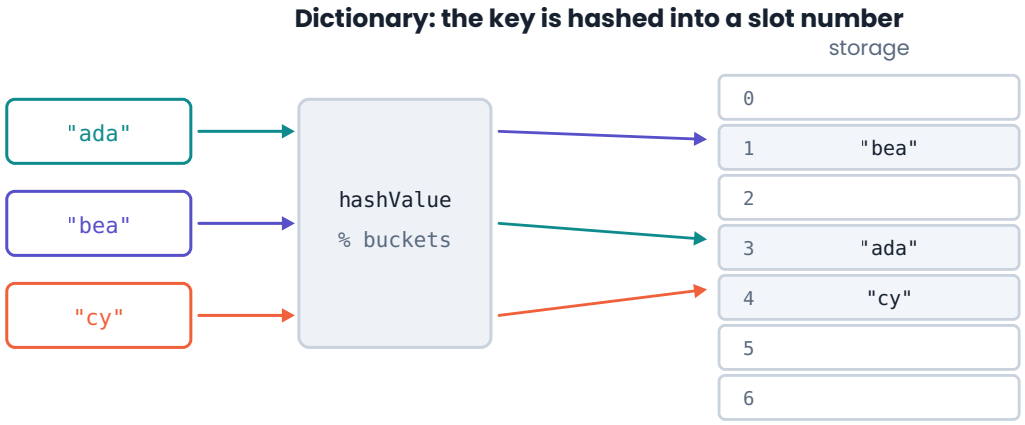
```
people.sorted { $0.age < $1.age }                       // ascending by age
people.sorted { ($0.age, $0.name) < ($1.age, $1.name) } // then by name
intervals.sort { $0[0] < $1[0] }                        // by start — chapter 28
nums.sorted(by: >)                                      // descending
```

Swift's sort is **not documented as stable**. The current implementation is a modified timsort and does happen to preserve the order of equal elements, but that is an implementation detail the standard library explicitly declines to promise. If equal elements must keep their original order, sort by a tuple that includes the original index — and say why you are doing it.

Chapter eleven

Dictionaries and sets

Both are hash tables. Both trade memory for the ability to answer "have I seen this" in constant time, and that trade is the backbone of the majority of optimal solutions.



No scanning. The key computes its own address, so lookup is O(1) average.

The key computes its own address, so there is nothing to scan.

The trade, stated plainly

The brute force for an enormous number of problems is a nested loop: for each element, look through the others. The optimisation is almost always the same — put what you have already seen into a dictionary or set, so the inner loop becomes a single lookup. $O(n^2)$ becomes $O(n)$, and you pay $O(n)$ memory for it.

Two Sum, the canonical example

```
func twoSum(_ nums: [Int], _ target: Int) -> [Int] {
    var seen: [Int: Int] = [:]          // value -> index
    for (i, n) in nums.enumerated() {
        if let j = seen[target - n] {   // have I already passed its partner?
            return [j, i]
        }
        seen[n] = i
    }
    return []
}
```

Cost $O(n)$ time, $O(n)$ space. The brute force is $O(n^2)$ time and $O(1)$ space. Name both, then say which you are choosing and why.

Counting, grouping, and mapping

Three idioms cover nearly every use:

```
// count occurrences
var counts: [Character: Int] = [:]
for c in text { counts[c, default: 0] += 1 }

// group by a key — anagram problems live here
var groups: [String: [String]] = [:]
for word in words {
    let key = String(word.sorted())
    groups[key, default: []].append(word)
}

// remember where you saw something
var lastIndex: [Character: Int] = [:]
for (i, c) in chars.enumerated() { lastIndex[c] = i }
```

The `default:` subscript does the "create it if missing, then modify" dance in one step, and it works for arrays as well as numbers. Learn the second example above by heart; grouping by a canonical key is a whole family of problems.

Sets

```
var seen: Set<Int> = []
for n in nums {
    if !seen.insert(n).inserted {    // insert returns whether it was new
        return n                     // first duplicate
    }
}
```

`insert` returning a tuple is a small piece of Swift trivia that reads very well when used deliberately.

> **Hashing is average-case**
>
> O(1) for a hash lookup is an average. Worst case, with every key colliding, it is O(n). Swift seeds its hashing randomly per process, so an adversarial input is not a practical concern — but if an interviewer asks "what is the worst case", the honest answer is O(n) and the reason is collisions.

Order is not defined

Iterating a dictionary or a set gives no guaranteed order, and the order can differ between runs of the same program. If you need order, sort at the end, or keep a separate array of keys as you go.

```
for key in counts.keys.sorted() { }
counts.sorted { $0.value > $1.value }.prefix(3)     // top 3 by count
```

Custom types as keys

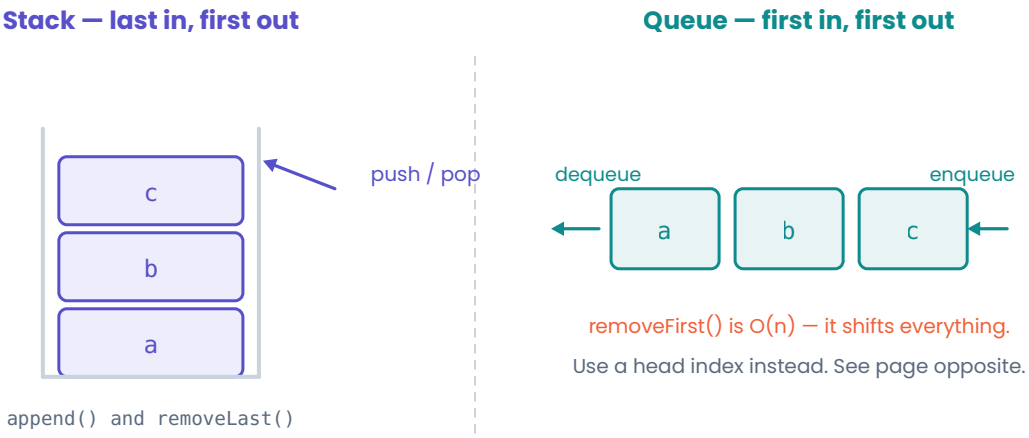
Any type can be a key if it is `Hashable`. Swift synthesises the conformance if you ask and all the stored properties are themselves `Hashable`:

```
struct Cell: Hashable { let row: Int, col: Int }
var visited: Set<Cell> = []
visited.insert(Cell(row: 0, col: 0))
```

Chapter twelve

Stacks and queues

Neither exists in the Swift standard library. Both are three lines on top of an array — but the queue has a trap in it that costs an entire order of complexity.



Same underlying array, opposite ends.

Stack

An array already is a stack. `append` and `removeLast` are both O(1), and they are the two operations you need.

In an interview, just use an array

```
var stack: [Int] = []
stack.append(1)
stack.append(2)
stack.removeLast()      // 2
stack.last              // Optional(1) — peek, without removing
stack.isEmpty
```

If you want the vocabulary to be explicit, wrapping it costs nothing:

```
struct Stack<T> {
    private var storage: [T] = []

    var isEmpty: Bool { storage.isEmpty }
    var count: Int { storage.count }
    var top: T? { storage.last }

    mutating func push(_ item: T) { storage.append(item) }
    mutating func pop() -> T? { storage.popLast() }
}
```

`popLast()` is the Optional-returning sibling of `removeLast()`, which traps on an empty array. Prefer it in library code.

Queue — and the mistake almost everyone makes

The obvious queue uses `append` to enqueue and `removeFirst()` to dequeue. It is also wrong, because `removeFirst()` shifts every remaining element left. Inside a BFS loop, that turns an O(V + E) traversal into $O(V^2)$.

> **removeFirst() is O(n)**
> This is the single most consequential performance trap in Swift interview code, precisely because it looks symmetrical with `removeLast()` and is not. If you write BFS with `removeFirst()`, expect the follow-up question.

The fix is to stop removing. Keep a head index and walk it forward, reclaiming the wasted prefix occasionally:

A queue with O(1) dequeue

```swift
struct Queue<T> {
    private var storage: [T] = []
    private var head = 0

    var isEmpty: Bool { head >= storage.count }
    var count: Int { storage.count - head }
    var front: T? { isEmpty ? nil : storage[head] }

    mutating func enqueue(_ item: T) {
        storage.append(item)
    }

    mutating func dequeue() -> T? {
        guard head < storage.count else { return nil }
        let item = storage[head]
        head += 1
        // once the dead prefix is more than half the buffer, compact it
        if head > 32 && head * 2 >= storage.count {
            storage.removeFirst(head)
            head = 0
        }
        return item
    }
}
```

Cost enqueue O(1) amortized · dequeue O(1) amortized · space O(n). The compaction step is O(n) but happens after at least n/2 dequeues, so it averages out to O(1) — the same argument as array doubling.

The shortcut when you are short on time

If you are pressed and BFS is not the point of the question, use an array with an index and skip the wrapper entirely. It is honest, obviously correct, and fast:

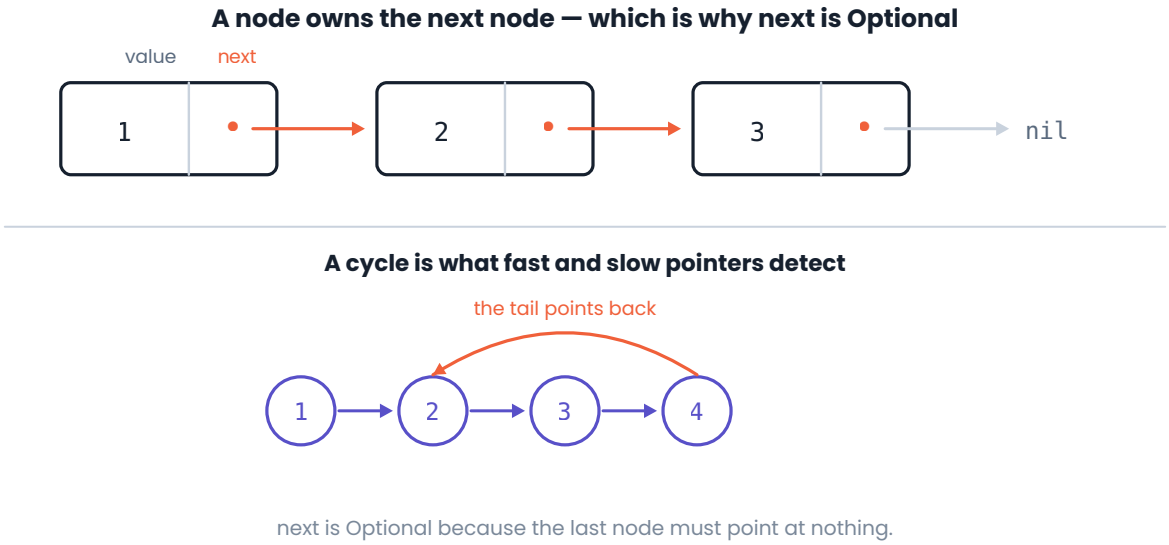
```swift
var queue = [start]
var head = 0
while head < queue.count {
    let node = queue[head]
    head += 1
    for next in neighbours(of: node) {
        queue.append(next)
    }
}
```

Say what you are doing — "I'll use an index instead of `removeFirst` so dequeue stays O(1)" — and you have answered the follow-up before it was asked.

Chapter thirteen

Linked lists

Rarely useful in production Swift, reliably present in interviews. They test pointer discipline: can you rearrange references without losing the rest of the list.



next is Optional because the last node must point at nothing.

The node

```
final class ListNode {
    var value: Int
    var next: ListNode?

    init(_ value: Int, _ next: ListNode? = nil) {
        self.value = value
        self.next = next
    }
}
```

It must be a `class`. A struct cannot contain itself — the compiler cannot compute a finite size — so reference semantics are not a choice here. `final` is a small performance courtesy and a signal you know what it does.

Walking

```
var current: ListNode? = head
while let node = current {
    print(node.value)
    current = node.next
}
```

Reversing — the one to know cold

Three pointers, one pass. Draw it once, then practise until you can write it without drawing.

```swift
func reverse(_ head: ListNode?) -> ListNode? {
    var previous: ListNode? = nil
    var current = head

    while let node = current {
        let next = node.next    // remember the rest before you cut
        node.next = previous    // flip this link backwards
        previous = node         // shuffle both pointers forward
        current = next
    }
    return previous             // previous is the new head
}
```

Cost O(n) time, O(1) space. The recursive version is also O(n) time but O(n) stack space — offer it as a variant, not as the answer.

The dummy head

When the answer's head might be a node you have not chosen yet — merging two lists, removing the first element, partitioning — allocate a throwaway node in front and return `dummy.next`. It removes every special case for the first element.

Merging two sorted lists

```swift
func merge(_ a: ListNode?, _ b: ListNode?) -> ListNode? {
    let dummy = ListNode(0)
    var tail = dummy
    var first = a, second = b

    while let x = first, let y = second {
        if x.value <= y.value {
            tail.next = x
            first = x.next
        } else {
            tail.next = y
            second = y.next
        }
        tail = tail.next!
    }
    tail.next = first ?? second     // attach whatever is left
    return dummy.next
}
```

Two habits that prevent most linked-list bugs

- Before you overwrite a `next`, save it. Losing the tail is the classic failure.
- Ask about the empty list and the single-node list out loud before you start. Half the edge cases in these problems are one of those two.

Finding the middle, and detecting a cycle

Both come from the same trick — two pointers at different speeds — which is important enough to get its own chapter in part three.

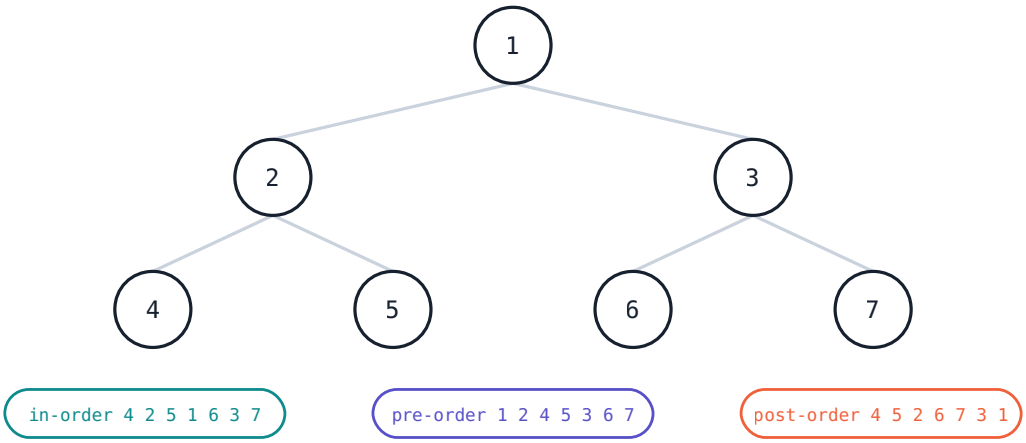
```
func middle(_ head: ListNode?) -> ListNode? {
    var slow = head, fast = head
    while fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
    }
    return slow                 // for even counts, the second middle
}
```

Chapter fourteen

Trees

A tree is a linked list that branches. Almost every tree question is a traversal wearing a disguise, so learn the three traversals first and the problems get much smaller.

One tree, three orders — the only difference is when you visit the node



In-order on a binary search tree returns the values already sorted.

Pre-, in- and post-order differ only in where the visit sits relative to the two recursive calls.

The node

```
final class TreeNode {
    var value: Int
    var left: TreeNode?
    var right: TreeNode?

    init(_ value: Int) { self.value = value }
}
```

The three depth-first traversals

They are the same function with one line moved.

```swift
func inorder(_ node: TreeNode?, _ out: inout [Int]) {
    guard let node else { return }
    inorder(node.left, &out)
    out.append(node.value)          // visit between the children
    inorder(node.right, &out)
}

func preorder(_ node: TreeNode?, _ out: inout [Int]) {
    guard let node else { return }
    out.append(node.value)          // visit before
    preorder(node.left, &out)
    preorder(node.right, &out)
}

func postorder(_ node: TreeNode?, _ out: inout [Int]) {
    guard let node else { return }
    postorder(node.left, &out)
    postorder(node.right, &out)
    out.append(node.value)          // visit after
}
```

In-order on a binary search tree returns the values in sorted order. **Pre-order** is the one to use when you need to copy or serialise a tree. **Post-order** is the one for anything where a node's answer depends on its children — heights, sums, deletions.

Height and the shape of recursive tree answers

Most tree problems fit this shape: solve the children, combine, return.

```swift
func height(_ node: TreeNode?) -> Int {
    guard let node else { return 0 }
    return 1 + max(height(node.left), height(node.right))
}
```

Is the tree balanced? — height, with an early exit

```swift
func isBalanced(_ root: TreeNode?) -> Bool {
    func check(_ node: TreeNode?) -> Int {     // -1 means "already unbalanced"
        guard let node else { return 0 }
        let left = check(node.left)
        if left == -1 { return -1 }
        let right = check(node.right)
        if right == -1 { return -1 }
        if abs(left - right) > 1 { return -1 }
        return 1 + max(left, right)
    }
    return check(root) != -1
}
```

Cost O(n) time — each node is visited once. O(h) space for the call stack, where h is the height: O(log n) for a balanced tree, O(n) for a degenerate one. Saying "O(h), which is O(n) in the worst case" is the complete answer.

Binary search trees

A BST adds one invariant: everything in the left subtree is smaller, everything in the right is larger. That turns search into binary search.

```swift
func search(_ root: TreeNode?, _ target: Int) -> TreeNode? {
    var node = root
    while let current = node {
        if current.value == target { return current }
        node = target < current.value ? current.left : current.right
    }
    return nil
}
```

Validating a BST is not a local check

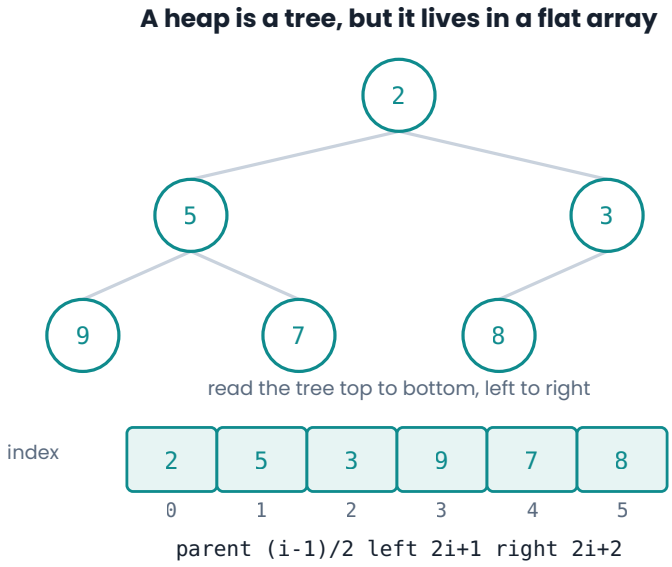
Comparing each node only against its two children is wrong — a value can be larger than its parent and still violate the rule further up the tree. Carry a valid range down instead, narrowing it at each step.

```swift
func isValidBST(_ root: TreeNode?) -> Bool {
    func check(_ node: TreeNode?, _ low: Int?, _ high: Int?) -> Bool {
        guard let node else { return true }
        if let low, node.value <= low { return false }
        if let high, node.value >= high { return false }
        return check(node.left, low, node.value)
            && check(node.right, node.value, high)
    }
    return check(root, nil, nil)
}
```

Chapter fifteen

Heaps and priority queues

A heap answers one question fast: what is the smallest thing left. Swift does not ship one, so this is the data structure you are most likely to be asked to write from scratch.



The tree is conceptual. The storage is a plain array, and the index arithmetic does the rest.

The idea

A binary heap is a complete tree with one rule: every parent is smaller than both of its children (for a min-heap). That is weaker than sorting — siblings are unordered — and the weakness is why insert and remove cost O(log n) instead of O(n).

Because the tree is complete, it packs into an array with no gaps, and the parent–child links become arithmetic:

Index arithmetic `left child of i = 2i + 1` · `right child of i = 2i + 2` · `parent of i = (i − 1) / 2`

The implementation

Two operations restore the rule. `siftUp` walks a too-small element towards the root after an insert; `siftDown` walks a too-large element towards the leaves after a removal.

A generic heap you can write in about five minutes

```
struct Heap<Element> {
    private var storage: [Element] = []
    private let isOrderedBefore: (Element, Element) -> Bool

    init(sort: @escaping (Element, Element) -> Bool) {
        isOrderedBefore = sort
    }

    var isEmpty: Bool { storage.isEmpty }
    var count: Int { storage.count }
    var peek: Element? { storage.first }

    mutating func insert(_ value: Element) {
        storage.append(value)
        siftUp(from: storage.count - 1)
    }

    mutating func remove() -> Element? {
        guard !storage.isEmpty else { return nil }
        storage.swapAt(0, storage.count - 1)   // move the root to the end
        let removed = storage.removeLast()     // ...and drop it, O(1)
        siftDown(from: 0)                      // then repair from the top
        return removed
    }

    private mutating func siftUp(from index: Int) {
        var child = index
        var parent = (child - 1) / 2
        while child > 0 && isOrderedBefore(storage[child], storage[parent]) {
            storage.swapAt(child, parent)
            child = parent
            parent = (child - 1) / 2
        }
    }

    private mutating func siftDown(from index: Int) {
        var parent = index
        while true {
            let left = 2 * parent + 1
            let right = 2 * parent + 2
            var candidate = parent
            if left < storage.count,
               isOrderedBefore(storage[left], storage[candidate]) {
                candidate = left
            }
            if right < storage.count,
               isOrderedBefore(storage[right], storage[candidate]) {
                candidate = right
            }
            if candidate == parent { return }   // rule restored
            storage.swapAt(parent, candidate)
            parent = candidate
        }
    }
}
```

Using it

```
var minHeap = Heap<Int>(sort: <)     // smallest comes out first
var maxHeap = Heap<Int>(sort: >)     // largest comes out first

for n in [5, 1, 4] { minHeap.insert(n) }
minHeap.remove()      // Optional(1)
minHeap.peek          // Optional(4)
```

Passing the comparison in as a closure is what makes one type serve as both a min-heap and a max-heap, and it lets you heap tuples or structs by any field:

```
var byDistance = Heap<(id: Int, dist: Double)>(sort: { $0.dist < $1.dist })
```

Cost insert O(log n) · remove O(log n) · peek O(1) · build from an array O(n) if you sift down from the middle backwards, O(n log n) if you insert one at a time.

When to reach for it

The signal is "k largest", "k smallest", "k most frequent", "median so far", or any repeated "give me the next smallest". If you only need the k best out of n and k is much smaller than n, a heap of size k gives you O(n log k), which beats sorting the whole thing at O(n log n). That comparison is chapter 26.

Ask before you build

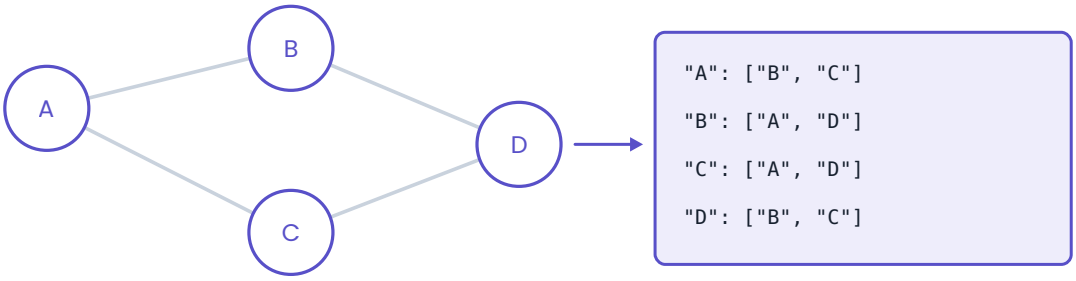
"Swift doesn't have a built-in priority queue — would you like me to write one, or may I assume it exists?" Most interviewers will let you assume it and move on to the actual problem, and the question itself shows you know the gap is there. If they ask you to write it, you have the twenty lines above.

Chapter sixteen

Graphs

A graph is nodes and the connections between them. In Swift it is almost always a dictionary from node to a list of neighbours, and building that dictionary is usually the first thing you do.

A graph is just a dictionary of neighbour lists



Undirected means the edge appears in both lists. Directed means it appears in one.

Almost every graph question in an interview starts by building this dictionary.

Adjacency list. Almost never a matrix, unless the graph is dense or the problem hands you one.

Representations

```swift
// adjacency list — the default
var graph: [Int: [Int]] = [:]

// from an edge list, undirected
for edge in edges {
    graph[edge[0], default: []].append(edge[1])
    graph[edge[1], default: []].append(edge[0])   // omit this line if directed
}

// adjacency matrix — only when the graph is dense or given to you this way
var matrix = Array(repeating: Array(repeating: false, count: n), count: n)
```

A grid is a graph too. The nodes are cells and the edges are the four (or eight) neighbours, which is why grid problems and graph problems use identical code once you have a `neighbours` function.

The two traversals

Both visit every reachable node once. The difference is the order, and the order is decided entirely by the container.

Depth-first, recursive

```swift
func dfs(_ node: Int, _ graph: [Int: [Int]], _ visited: inout Set<Int>) {
    guard !visited.contains(node) else { return }
    visited.insert(node)
    for next in graph[node] ?? [] {
        dfs(next, graph, &visited)
    }
}
```

Breadth-first, with an index-based queue

```
func bfs(from start: Int, _ graph: [Int: [Int]]) -> [Int] {
    var visited: Set<Int> = [start]
    var queue = [start]
    var head = 0
    var order: [Int] = []

    while head < queue.count {
        let node = queue[head]
        head += 1
        order.append(node)
        for next in graph[node] ?? [] where !visited.contains(next) {
            visited.insert(next)      // mark on enqueue, not on dequeue
            queue.append(next)
        }
    }
    return order
}
```

Mark visited when you enqueue

If you wait until you dequeue, the same node can be pushed several times before it is first processed, and the queue can blow up. Insert into `visited` in the same breath as `append`.

Which one to use

Use **BFS** when the question is about the shortest path or fewest steps in an unweighted graph — BFS reaches every node by the fewest edges. Use **DFS** for reachability, cycle detection, connected components, topological order, and anything where you need to explore one branch fully before judging it.

Counting connected components

The shape of an enormous number of grid problems — islands, regions, provinces — is a loop over every cell that starts a fresh traversal whenever it finds something unvisited.

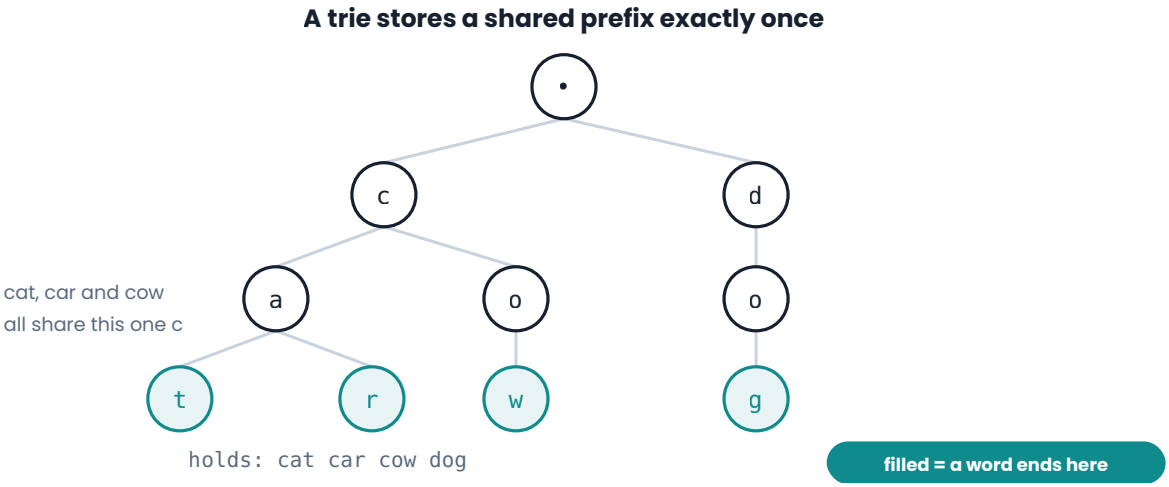
```
func countIslands(_ grid: [[Character]]) -> Int {
    var grid = grid                                  // local mutable copy
    let rows = grid.count, cols = grid[0].count
    var count = 0

    func sink(_ r: Int, _ c: Int) {
        guard r >= 0, r < rows, c >= 0, c < cols, grid[r][c] == "1" else { return }
        grid[r][c] = "0"                             // mark by overwriting
        sink(r + 1, c); sink(r - 1, c)
        sink(r, c + 1); sink(r, c - 1)
    }

    for r in 0..<rows {
        for c in 0..<cols where grid[r][c] == "1" {
            count += 1
            sink(r, c)
        }
    }
    return count
}
```

Cost O(rows × cols) time and, worst case, the same for the recursion stack. Overwriting the grid avoids a separate visited set; say out loud that you are mutating the input, and offer the non-destructive version if that matters.

Chapter seventeen

Tries

A tree of characters where the path from the root spells a prefix. Narrow in application, but when a problem says "prefix", this is the entire answer.



Every node is a prefix. A flag marks which prefixes are also complete words.

```swift
final class TrieNode {
    var children: [Character: TrieNode] = [:]
    var isWord = false
}

final class Trie {
    private let root = TrieNode()

    func insert(_ word: String) {
        var node = root
        for c in word {
            if node.children[c] == nil {
                node.children[c] = TrieNode()
            }
            node = node.children[c]!
        }
        node.isWord = true
    }

    func search(_ word: String) -> Bool {
        find(word)?.isWord ?? false
    }

    func startsWith(_ prefix: String) -> Bool {
        find(prefix) != nil
    }

    private func find(_ prefix: String) -> TrieNode? {
        var node = root
        for c in prefix {
            guard let next = node.children[c] else { return nil }
            node = next
        }
        return node
    }
}
```

Cost insert, search and prefix check are all O(k) where k is the length of the word — independent of how many words are stored. Space is O(total characters), less the shared prefixes.

That independence from the number of stored words is the whole point. A set of strings can answer "is this an exact word" in O(k) too, but it cannot answer "does any word start with this" without checking all of them.

When it earns its keep

Autocomplete, word search on a board, and any problem where you are matching many words against the same text at once. In the last case a trie lets you walk all candidate words simultaneously, which is what makes the board-search problems tractable.

Part three

Patterns — the part that changes your odds

There are not a thousand problems. There are about fourteen shapes, and a thousand costumes. This part is about seeing the shape in the first two minutes.

Chapter eighteen

Choosing a pattern

The first two minutes decide the next forty. Before writing anything, read the input for the signals below — they are far more reliable than the story the problem is wrapped in.

Read the input, pick the pattern

sorted array **two pointers / binary search**	count the ways / best value **dynamic programming**
contiguous subarray **sliding window**	shortest path, equal steps **BFS**
linked list, no extra space **fast & slow pointers**	next greater / smaller **monotonic stack**
k largest / smallest **heap**	connected groups **union-find**
all combinations **backtracking**	many range sums **prefix sums**

If two fit, say both out loud and pick the simpler one.

The mapping is not perfect, but it is right often enough to be worth memorising.

What to look at, in order

The shape of the input. Sorted array, unsorted array, linked list, tree, grid, graph, string, single number. This alone eliminates most patterns.

What is being asked for. A single value, an index, a count, every possible arrangement, the best of something, whether something is possible. "Every possible" means backtracking. "Best" usually means dynamic programming or greedy.

The constraints. These are a hint, not decoration. $n \le 20$ means exponential is fine and probably intended. $n \le 10^3$ tolerates $O(n^2)$. $n \le 10^5$ demands $O(n \log n)$ or better. $n \le 10^9$ means the answer cannot involve visiting the input at all — think maths, or binary search on the answer.

> **Constraints are the strongest signal in the problem**
>
> An interviewer who says "the array can have up to a hundred thousand elements" has just told you that $O(n^2)$ is wrong. Read constraints out loud and convert them into a target complexity before you design anything. It reframes the whole conversation from "find an answer" to "find an answer that fits".

Say the brute force first, always

State the obvious solution and its complexity in one sentence, then improve on it. This costs fifteen seconds and buys you three things: a correct answer already on the table, a baseline

to measure the optimisation against, and visible evidence that you are reasoning rather than reciting.

"The brute force is to check every pair, which is $O(n^2)$. I think I can get that to $O(n)$ with a dictionary — let me try."

The order of operations

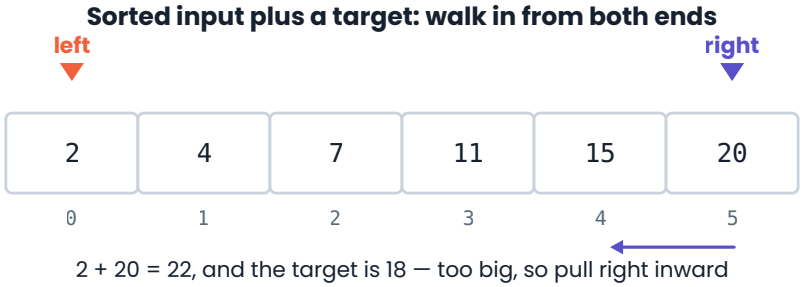
1. Restate the problem and confirm the input and output types.
2. Ask about edge cases — empty, one element, duplicates, negatives.
3. Name the brute force and its complexity.
4. Name the pattern you think applies, and why.
5. Write the code, talking through it.
6. Trace one small example by hand.
7. State the final complexity and one thing you would change with more time.

Steps 6 and 7 get skipped under pressure and are where a surprising number of interviews are won.

Chapter nineteen

Two pointers

The signal A sorted array, or a problem about pairs, or a palindrome. Anything where you would otherwise write two nested loops over the same array.



Too big? Shrink from the right. Too small? Grow from the left.

One move rules out a whole row or column of pairs, so O(n) replaces the O(n²) double loop.

Each comparison rules out an entire row or column of the pairs you would otherwise check.

Why it works

The brute force checks every pair: O(n²). When the array is sorted, one comparison tells you which direction to move, because you know everything to the right is bigger and everything to the left is smaller. Each step permanently eliminates a whole set of candidates, so a single pass suffices.

Template — converging from both ends

```
func twoSumSorted(_ nums: [Int], _ target: Int) -> [Int] {
    var left = 0, right = nums.count - 1

    while left < right {
        let sum = nums[left] + nums[right]
        if sum == target { return [left, right] }
        if sum < target { left += 1 }        // need bigger, move left up
        else { right -= 1 }                  // need smaller, move right down
    }
    return []
}
```

Template — same direction, one slow and one fast

The other half of this pattern uses both pointers moving forward at different rates. It is how you remove or filter in place, without allocating a second array.

Remove duplicates from a sorted array, in place

```swift
func removeDuplicates(_ nums: inout [Int]) -> Int {
    guard !nums.isEmpty else { return 0 }
    var write = 1                              // where the next keeper goes

    for read in 1..<nums.count {
        if nums[read] != nums[write - 1] {
            nums[write] = nums[read]
            write += 1
        }
    }
    return write                               // new logical length
}
```

The `write` pointer only advances when something is kept, so everything before it is always the answer so far. This "read and write index" shape appears constantly — moving zeroes, filtering, compacting.

Worked example: three sum

Sort, fix one element, and two-pointer the rest. The only real difficulty is skipping duplicates.

```swift
func threeSum(_ nums: [Int]) -> [[Int]] {
    let nums = nums.sorted()
    var result: [[Int]] = []

    for i in 0..<nums.count {
        if nums[i] > 0 { break }                       // sorted: no triple can reach 0
        if i > 0 && nums[i] == nums[i - 1] { continue } // skip a repeated anchor

        var left = i + 1, right = nums.count - 1
        while left < right {
            let sum = nums[i] + nums[left] + nums[right]
            if sum < 0 {
                left += 1
            } else if sum > 0 {
                right -= 1
            } else {
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right && nums[left] == nums[left - 1] { left += 1 }
                while left < right && nums[right] == nums[right + 1] { right -= 1 }
            }
        }
    }
    return result
}
```

Cost O(n log n) for the sort plus O(n²) for the nested scan, so O(n²) overall. O(1) extra space if you ignore the sort's. The brute force is O(n³).

Duplicate handling is the whole difficulty

Skipping the repeated anchor and the repeated inner values are two separate jobs. Miss either and you emit the same triple more than once. When you practise this problem, practise it on an input full of duplicates — `[0,0,0,0]` is the test that catches it.

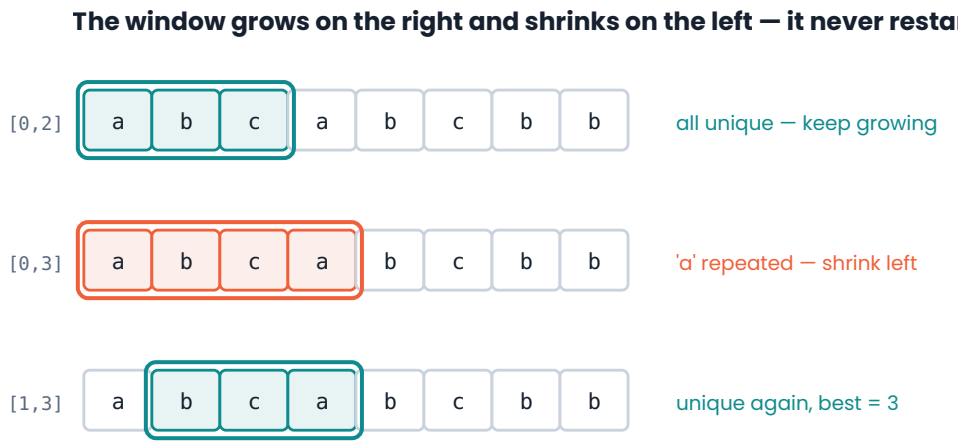
Drill these

Valid palindrome · Two sum on a sorted array · Container with most water · Three sum · Move zeroes · Remove duplicates from sorted array · Sort colours · Trapping rain water

Chapter twenty

Sliding window

> **The signal** A **contiguous** subarray or substring, plus a word like longest, shortest, or exactly-k. If the elements do not have to be adjacent, this is not the pattern.



Every index enters once and leaves once, so the whole scan is O(n).

The window never resets, which is what keeps the whole scan linear.

Why it works

The brute force examines every window: $O(n^2)$ of them, each costing something to evaluate. But adjacent windows overlap almost entirely. Instead of recomputing, you extend on the right and, when the window becomes invalid, contract from the left. Every index enters once and leaves once, so the total work is O(n) regardless of how the window moves.

Template — variable window

The shape of nearly every window problem

```
func longestValidWindow(_ items: [Int]) -> Int {
    var left = 0
    var best = 0
    // ... whatever state describes the current window

    for right in 0..<items.count {
        // 1. extend: add items[right] to the window state

        while /* the window is invalid */ false {
            // 2. contract: remove items[left] from the state
            left += 1
        }

        // 3. record: the window [left...right] is valid here
        best = max(best, right - left + 1)
    }
    return best
}
```

Three slots: extend, contract, record. Every problem in this family fills those three slots differently, and the skeleton never changes.

Worked example: longest substring without repeating characters

```swift
func lengthOfLongestSubstring(_ s: String) -> Int {
    let chars = Array(s)
    var lastSeen: [Character: Int] = [:]
    var left = 0, best = 0

    for right in 0..<chars.count {
        let c = chars[right]
        // if we have seen it inside the current window, jump left past it
        if let previous = lastSeen[c], previous >= left {
            left = previous + 1
        }
        lastSeen[c] = right
        best = max(best, right - left + 1)
    }
    return best
}
```

The `previous >= left` check is the part people drop. Without it, a character last seen *before* the window would drag `left` backwards, and the window would grow when it should not.

Cost O(n) time — each index is visited once by `right` and at most once by `left`. O(k) space, where k is the size of the character set.

Template — fixed window

When the window size is given, the code is simpler: add one, drop one, no while loop.

Maximum sum of any k consecutive elements

```swift
func maxSum(_ nums: [Int], _ k: Int) -> Int {
    guard nums.count >= k else { return 0 }
    var sum = nums[0..<k].reduce(0, +)
    var best = sum

    for right in k..<nums.count {
        sum += nums[right] - nums[right - k]   // in on the right, out on the left
        best = max(best, sum)
    }
    return best
}
```

Negative numbers break the shrink condition

Variable-size windows on sums assume that adding an element makes the sum bigger, so shrinking always helps. With negative numbers that is false, and the sliding window is simply the wrong tool — reach for prefix sums with a dictionary instead (chapter 30). Check the constraints for "positive integers" before you commit.

Drill these

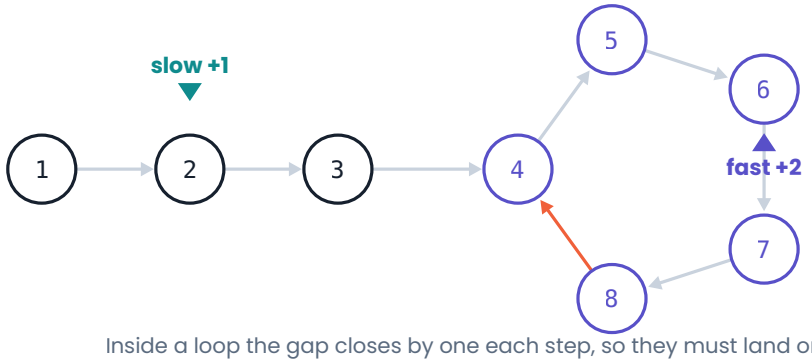
Longest substring without repeating characters · Maximum average subarray · Longest repeating character replacement · Permutation in string · Minimum window substring · Fruit into baskets · Subarray product less than k

Chapter twenty-one

Fast and slow pointers

The signal A linked list, plus a requirement of constant extra space. Also anything phrased as a cycle, a loop, or "the middle".

Two speeds on one list: the fast pointer laps the slow one



Inside a loop the gap closes by one each step, so they must land on the same node.
No loop? The fast pointer runs off the end and returns nil.

Inside a loop the gap between the two closes by one each step, so a meeting is guaranteed.

Why it works

Move one pointer one step and another two steps. If the list ends, the fast one falls off and there is no cycle. If it does not end, both are trapped in the loop, and since the fast pointer gains exactly one position per step, it must eventually land on the slow one. No set, no extra memory.

Template — cycle detection

```swift
func hasCycle(_ head: ListNode?) -> Bool {
    var slow = head, fast = head

    while fast != nil && fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
        if slow === fast { return true }     // === compares identity, not value
    }
    return false
}
```

`===` is reference identity. Using `==` here would compare values and report a false cycle on any list with two equal elements.

Template — the middle

```
func middleNode(_ head: ListNode?) -> ListNode? {
    var slow = head, fast = head
    while fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
    }
    return slow
}
```

The loop condition decides which middle you get for even-length lists. `while fast?.next != nil` gives the first of the two; `while fast != nil && fast?.next != nil` gives the second. Know which one your problem wants — palindrome checks care.

Worked example: palindrome linked list

Find the middle, reverse the second half, compare, and (politely) restore.

```
func isPalindrome(_ head: ListNode?) -> Bool {
    guard head?.next != nil else { return true }

    // 1. middle
    var slow = head, fast = head
    while fast?.next != nil {
        slow = slow?.next
        fast = fast?.next?.next
    }

    // 2. reverse from the middle
    var previous: ListNode? = nil
    var current = slow
    while let node = current {
        let next = node.next
        node.next = previous
        previous = node
        current = next
    }

    // 3. walk both halves inwards
    var front = head, back = previous
    while let f = front, let b = back {
        if f.value != b.value { return false }
        front = f.next
        back = b.next
    }
    return true
}
```

Cost O(n) time, O(1) space. The obvious solution — copy the values into an array and two-pointer it — is O(n) space, and offering that first as the baseline is the right move.

Where the cycle starts

A likely follow-up: once slow and fast meet, move one pointer back to the head and advance both one step at a time. They meet again exactly at the entrance to the cycle. The proof is a short piece of modular arithmetic; you do not need to derive it live, but you should know the result and be able to say it is a known property rather than pretending to have invented it.

Drill these

Linked list cycle · Linked list cycle II · Middle of the linked list · Palindrome linked list · Happy number · Find the duplicate number · Reorder list

Chapter twenty-two

Binary search

> **The signal** A sorted array. Or — the version people miss — a huge range of candidate answers where you can cheaply test "is this answer good enough".

Twenty steps covers a million elements. Thirty covers a billion.

The classic form

```swift
func binarySearch(_ nums: [Int], _ target: Int) -> Int {
    var low = 0, high = nums.count - 1

    while low <= high {
        let mid = low + (high - low) / 2       // avoids overflow on huge bounds
        if nums[mid] == target { return mid }
        if nums[mid] < target { low = mid + 1 }
        else { high = mid - 1 }
    }
    return -1
}
```

Two details carry all the bugs: `low <= high` (not `<`) when `high` starts at `count - 1`, and moving past `mid` rather than to it. Get those wrong and you loop forever.

The form you should actually memorise

Most real problems are not "find this exact value" but "find the first position where a condition becomes true". This version has no equality case, always terminates, and adapts to almost everything.

First index where the predicate is true

```
func firstTrue(_ range: Range<Int>, _ predicate: (Int) -> Bool) -> Int {
    var low = range.lowerBound
    var high = range.upperBound          // exclusive — one past the end

    while low < high {
        let mid = low + (high - low) / 2
        if predicate(mid) {
            high = mid                   // mid might be the answer, keep it
        } else {
            low = mid + 1                // mid is definitely not, discard it
        }
    }
    return low                           // == upperBound if nothing qualifies
}

// insertion point for a target in a sorted array
let index = firstTrue(0..<nums.count) { nums[$0] >= target }
```

Why this version is safer

The condition must be monotonic: false, false, false, true, true. Once you frame the problem that way, the loop is mechanical — `high = mid` keeps a candidate, `low = mid + 1` discards one, the range always shrinks, and it always ends at the boundary. There is no equality branch to get wrong.

Binary search on the answer

This is the version that separates candidates. When the problem asks for a minimum or maximum value, and checking a specific value is easy, search the answer space rather than the input.

Ship capacity: least capacity that finishes within `days` days

```
func shipWithinDays(_ weights: [Int], _ days: Int) -> Int {
    func canShip(_ capacity: Int) -> Bool {
        var daysNeeded = 1, load = 0
        for w in weights {
            if load + w > capacity {
                daysNeeded += 1
                load = 0
            }
            load += w
        }
        return daysNeeded <= days
    }

    var low = weights.max() ?? 0          // must fit the heaviest single item
    var high = weights.reduce(0, +)       // everything in one day
    while low < high {
        let mid = low + (high - low) / 2
        if canShip(mid) { high = mid } else { low = mid + 1 }
    }
    return low
}
```

Cost O(n log S), where S is the range of possible capacities. The `canShip` check is O(n) and runs log S times.

The tell for this variant is a question of the form "minimum largest..." or "maximum smallest...", combined with constraints too large to enumerate. If checking one candidate is easy and the answer is monotonic, binary search it.

Rotated sorted arrays

One half is always still sorted. Work out which half, then decide whether the target is inside it.

```
func searchRotated(_ nums: [Int], _ target: Int) -> Int {
    var low = 0, high = nums.count - 1

    while low <= high {
        let mid = low + (high - low) / 2
        if nums[mid] == target { return mid }

        if nums[low] <= nums[mid] {                       // left half is sorted
            if nums[low] <= target && target < nums[mid] { high = mid - 1 }
            else { low = mid + 1 }
        } else {                                          // right half is sorted
            if nums[mid] < target && target <= nums[high] { low = mid + 1 }
            else { high = mid - 1 }
        }
    }
    return -1
}
```

Drill these

Binary search · Search insert position · First bad version · Find minimum in rotated sorted array · Search in rotated sorted array · Koko eating bananas · Capacity to ship packages · Median of two sorted arrays

Chapter twenty-three

Depth-first search

> **The signal** A tree, a graph, or a grid, and a question about reachability, structure, or "explore everything from here".

The shape

DFS commits to one branch and follows it to the end before backing up. Recursion gives you the stack for free, which is why the recursive version is almost always the one to write.

The three lines that never change

```swift
func dfs(_ node: Node) {
    guard !visited.contains(node) else { return }   // 1. have I been here
    visited.insert(node)                            // 2. mark it
    for next in neighbours(of: node) {              // 3. go deeper
        dfs(next)
    }
}
```

On a grid

Bounds and validity live in one guard at the top, so the recursive calls stay clean.

```swift
func dfs(_ grid: inout [[Character]], _ r: Int, _ c: Int) {
    guard r >= 0, r < grid.count,
          c >= 0, c < grid[0].count,
          grid[r][c] == "1" else { return }

    grid[r][c] = "0"                    // mark visited by mutating
    dfs(&grid, r + 1, c)
    dfs(&grid, r - 1, c)
    dfs(&grid, r, c + 1)
    dfs(&grid, r, c - 1)
}
```

Pushing the checks into the callee, rather than testing before each of the four calls, is the difference between four lines and sixteen.

Returning a value up the tree

When each node's answer depends on its children, compute the children first and combine on the way back up. This is post-order, and it is the most useful tree pattern there is.

Diameter: the longest path between any two nodes

```
func diameterOfBinaryTree(_ root: TreeNode?) -> Int {
    var best = 0

    @discardableResult
    func depth(_ node: TreeNode?) -> Int {
        guard let node else { return 0 }
        let left = depth(node.left)
        let right = depth(node.right)
        best = max(best, left + right)      // path through this node
        return 1 + max(left, right)         // depth reported to the parent
    }

    depth(root)
    return best
}
```

The trick worth internalising: the function returns one thing to its parent while updating a different, wider answer on the side. A large family of hard tree problems is exactly this.

Cost O(V + E) for a graph, O(n) for a tree, O(rows × cols) for a grid. Space is O(depth) for the call stack — worst case O(n) for a skewed tree or a snaking grid path.

Recursion depth is a real limit

A 10^5-node path will overflow the stack. If the constraints allow a graph that deep, say so and offer the iterative version with an explicit stack. Interviewers rarely require it, but naming the risk is worth a lot.

Iterative DFS

```
var stack = [start]
var visited: Set<Int> = []
while let node = stack.popLast() {
    guard !visited.contains(node) else { continue }
    visited.insert(node)
    for next in graph[node] ?? [] { stack.append(next) }
}
```

Note that this is BFS's code with one word changed — `popLast` instead of taking from the front. The container is the algorithm.

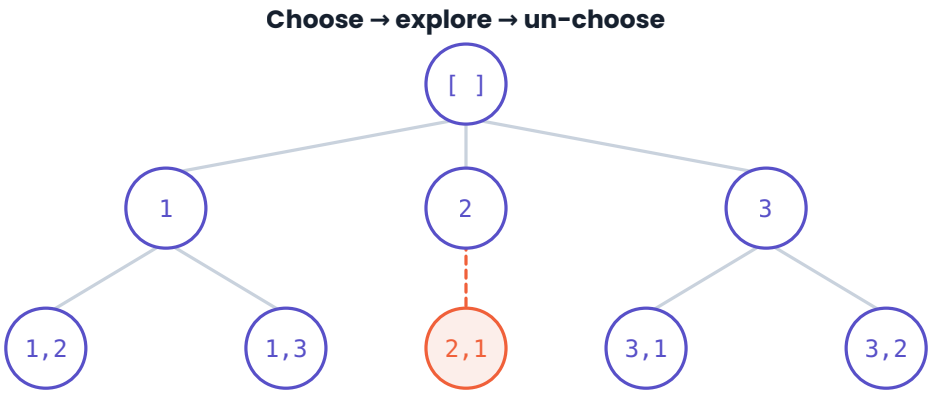
Drill these

Number of islands · Max area of island · Clone graph · Course schedule · Path sum · Diameter of binary tree · Lowest common ancestor · Pacific Atlantic water flow

Chapter twenty-four

Backtracking

> **The signal** "All", "every", "generate", "how many ways" — combined with a small n. Permutations, combinations, subsets, board placements.



The red branch broke a rule, so we undo the last choice and try the next one.

The path array is shared — pushing and popping is what makes it cheap.

Choose, explore, un-choose. The un-choose is the whole pattern.

The shape

Backtracking is DFS over a tree of decisions that you never actually build. You make a choice, recurse, and then undo the choice so the next branch starts clean. That undo is why one shared `path` array is enough for the entire search.

The template

```
func solve(_ input: [Int]) -> [[Int]] {
    var result: [[Int]] = []
    var path: [Int] = []

    func backtrack(_ start: Int) {
        if /* the path is a complete answer */ true {
            result.append(path)          // append copies — value semantics
        }
        for i in start..<input.count {
            // if this choice is invalid, continue

            path.append(input[i])        // choose
            backtrack(i + 1)             // explore
            path.removeLast()            // un-choose
        }
    }

    backtrack(0)
    return result
}
```

> **Why append does not need a copy**
>
> `result.append(path)` is safe even though `path` keeps mutating afterwards, because Swift arrays are value types — the append stores a copy. In a reference-semantics language you would need an explicit clone here, and forgetting it is a classic bug. In Swift the value semantics are doing you a favour.

Subsets

```
func subsets(_ nums: [Int]) -> [[Int]] {
    var result: [[Int]] = []
    var path: [Int] = []

    func backtrack(_ start: Int) {
        result.append(path)                    // every node is an answer
        for i in start..<nums.count {
            path.append(nums[i])
            backtrack(i + 1)                   // i + 1: never reuse an element
            path.removeLast()
        }
    }

    backtrack(0)
    return result
}
```

Permutations

Order matters, so every unused element is a candidate at every position — which means a `used` array instead of a start index.

```
func permute(_ nums: [Int]) -> [[Int]] {
    var result: [[Int]] = []
    var path: [Int] = []
    var used = Array(repeating: false, count: nums.count)

    func backtrack() {
        if path.count == nums.count {
            result.append(path)
            return
        }
        for i in 0..<nums.count where !used[i] {
            used[i] = true
            path.append(nums[i])
            backtrack()
            path.removeLast()
            used[i] = false                    // undo both
        }
    }

    backtrack()
    return result
}
```

> **Start index** for combinations and subsets, where order does not matter. **Used array** for permutations, where it does. Choosing the wrong one is the most common structural mistake in this pattern.

Pruning

Without pruning, backtracking is brute force with extra steps. The value comes from abandoning a branch the moment it cannot lead anywhere.

Combination sum: stop as soon as we overshoot

```swift
func combinationSum(_ candidates: [Int], _ target: Int) -> [[Int]] {
    let candidates = candidates.sorted()          // sorting enables the break
    var result: [[Int]] = []
    var path: [Int] = []

    func backtrack(_ start: Int, _ remaining: Int) {
        if remaining == 0 {
            result.append(path)
            return
        }
        for i in start..<candidates.count {
            if candidates[i] > remaining { break }   // sorted: the rest are worse
            path.append(candidates[i])
            backtrack(i, remaining - candidates[i])  // i, not i+1: reuse allowed
            path.removeLast()
        }
    }

    backtrack(0, target)
    return result
}
```

Cost Subsets $O(n \cdot 2^n)$ · permutations $O(n \cdot n!)$ · combination sum exponential in the target. The extra factor of n is the copy made when appending a completed path. Space is $O(n)$ for the path plus $O(n)$ for the recursion.

Drill these

Subsets · Subsets II · Permutations · Combination sum · Combination sum II · Palindrome partitioning · Word search · Letter combinations of a phone number · N-queens

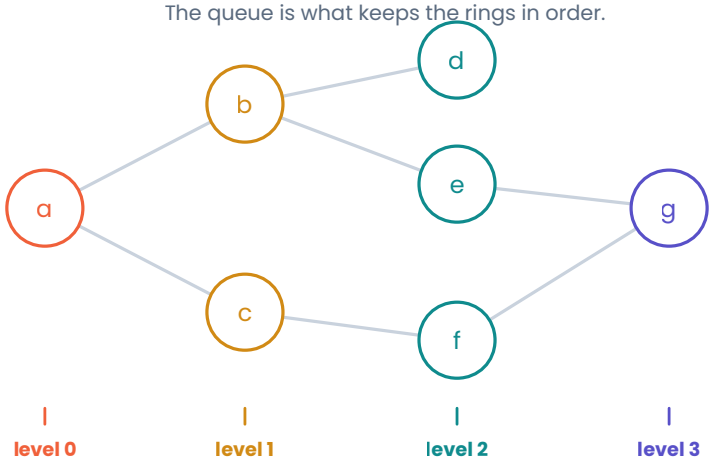
Chapter twenty-five

Breadth-first search

> **The signal** Shortest path, fewest steps, minimum number of moves — in an **unweighted** graph or grid. Also anything that asks about a tree level by level.

BFS finishes a whole ring before it starts the next one

The queue is what keeps the rings in order.



Everything at distance one is finished before anything at distance two begins.

Why it finds shortest paths

BFS reaches nodes in order of distance from the start. The first time you arrive at a node, you have arrived by the fewest possible edges — there is no shorter route still to come. That guarantee is what DFS lacks, and it holds only while every edge costs the same. Weighted edges need Dijkstra, which is BFS with a heap instead of a queue.

Template

```swift
func shortestPath(from start: Int, to goal: Int, _ graph: [Int: [Int]]) -> Int {
    if start == goal { return 0 }

    var visited: Set<Int> = [start]
    var queue = [start]
    var head = 0
    var steps = 0

    while head < queue.count {
        let levelSize = queue.count - head        // freeze this level's width
        steps += 1

        for _ in 0..<levelSize {
            let node = queue[head]
            head += 1

            for next in graph[node] ?? [] where !visited.contains(next) {
                if next == goal { return steps }
                visited.insert(next)
                queue.append(next)
            }
        }
    }
    return -1
}
```

The `levelSize` line is the one to understand. Capturing the level's width before processing it is what lets you count distance — without it you can still traverse, but you lose track of which ring you are on.

Level-order traversal of a tree

The same structure, returning the levels instead of counting them.

```
func levelOrder(_ root: TreeNode?) -> [[Int]] {
    guard let root else { return [] }
    var result: [[Int]] = []
    var queue = [root]
    var head = 0

    while head < queue.count {
        let levelSize = queue.count - head
        var level: [Int] = []

        for _ in 0..<levelSize {
            let node = queue[head]
            head += 1
            level.append(node.value)
            if let left = node.left { queue.append(left) }
            if let right = node.right { queue.append(right) }
        }
        result.append(level)
    }
    return result
}
```

Multi-source BFS

Start with several nodes in the queue at once and the algorithm expands from all of them simultaneously. Rotting oranges, distance-to-nearest-zero, and fire-spread problems are all this.

```
var queue: [(Int, Int)] = []
for r in 0..<rows {
    for c in 0..<cols where grid[r][c] == 2 {
        queue.append((r, c))          // every source starts at distance 0
    }
}
// ...then the usual level-by-level loop
```

Cost O(V + E) time, O(V) space for the queue and visited set. On a grid that is O(rows × cols) for both.

Two failure modes

- Using `removeFirst()` — O(n) per dequeue turns the whole traversal quadratic. Use a head index.
- Marking visited on dequeue instead of enqueue — the same node gets queued many times before it is first seen.

Drill these

Binary tree level order traversal · Rotting oranges · 01 matrix · Word ladder · Shortest path in binary matrix · Open the lock · Number of islands (BFS version)

Chapter twenty-six

Top-k with a heap

The signal "The k largest", "the k most frequent", "the k closest", "the median as elements arrive". Any time you need the best few out of many, but not the full ordering.

Why not just sort

Sorting gives you the answer at O(n log n). A heap of size k gives it at O(n log k). When k is small and n is large — the usual case — that is a real win, and more importantly it is the answer the interviewer is looking for.

Keep a heap of exactly **k** elements. To find the k **largest**, use a **min**-heap and evict the smallest whenever the heap grows past k. The inversion feels wrong the first time and is the whole trick: the root is the weakest survivor, so it is the one to throw away.

K largest elements

```
func kLargest(_ nums: [Int], _ k: Int) -> [Int] {
    var heap = Heap<Int>(sort: <)          // min-heap

    for n in nums {
        heap.insert(n)
        if heap.count > k {
            heap.remove()                  // drop the smallest so far
        }
    }

    var result: [Int] = []
    while let n = heap.remove() { result.append(n) }
    return result
}
```

Top k frequent elements

Count first, then heap the counts. This composes two patterns, which is typical of the harder versions.

```
func topKFrequent(_ nums: [Int], _ k: Int) -> [Int] {
    var counts: [Int: Int] = [:]
    for n in nums { counts[n, default: 0] += 1 }

    var heap = Heap<(value: Int, count: Int)>(sort: { $0.count < $1.count })
    for (value, count) in counts {
        heap.insert((value, count))
        if heap.count > k { heap.remove() }
    }

    var result: [Int] = []
    while let entry = heap.remove() { result.append(entry.value) }
    return result
}
```

Cost O(n) to count, O(n log k) to heap, O(k log k) to drain. Overall O(n log k) time and O(n) space for the count map. Mention bucket sort as the O(n) alternative — counts are bounded by n, so you can index by frequency directly.

Two heaps: the running median

A max-heap for the smaller half and a min-heap for the larger half. The median sits at the two roots.

```
struct MedianFinder {
    private var low = Heap<Int>(sort: >)    // max-heap: the smaller half
    private var high = Heap<Int>(sort: <)   // min-heap: the larger half

    mutating func add(_ n: Int) {
        low.insert(n)
        // everything in low must be <= everything in high
        if let lowTop = low.remove() { high.insert(lowTop) }
        // keep low the same size or one larger
        if high.count > low.count, let highTop = high.remove() {
            low.insert(highTop)
        }
    }

    var median: Double? {
        guard let lowTop = low.peek else { return nil }
        if low.count > high.count { return Double(lowTop) }
        guard let highTop = high.peek else { return nil }
        return Double(lowTop + highTop) / 2.0
    }
}
```

The unconditional push-then-pop through both heaps looks wasteful and is what makes the balancing correct without a pile of special cases.

Cost O(log n) per insertion, O(1) to read the median. Space O(n).

Drill these

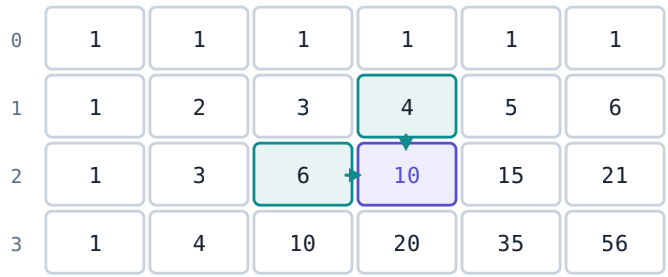
Kth largest element in an array · Top k frequent elements · K closest points to origin · Find median from data stream · Merge k sorted lists · Task scheduler · Reorganize string

Chapter twenty-seven

Dynamic programming

> **The signal** "How many ways", "the minimum or maximum cost", "is it possible to reach" — where a choice now affects what is available later, and the same subproblem keeps reappearing.

Each cell is answered once, from cells already answered




```
dp[r][c] = dp[r-1][c] + dp[r][c-1]
```

The recursion is the same formula. The table just refuses to compute anything twice.

The table is just a place to write down answers so you never compute one twice.

What it actually is

Dynamic programming is recursion plus memory. If your recursive solution asks the same question repeatedly, store the answer the first time. That is the entire idea; everything else is bookkeeping about where to store it and in what order to fill it.

Two conditions must hold. **Optimal substructure**: the best answer is built from best answers to smaller versions. **Overlapping subproblems**: those smaller versions recur. Without the second, you have plain recursion and memoisation buys nothing.

The four questions

Work through these in order, out loud, before writing code:

1. **What is the state?** What arguments fully describe a subproblem? This is the hard one — get it wrong and nothing else works.
2. **What is the recurrence?** How does one state's answer combine its children's?
3. **What are the base cases?** The smallest states, answered directly.
4. **In what order do I fill the table?** Every state must be computed before anything that depends on it.

Top-down: recursion with a cache

Closest to how you actually think, and the easier one to derive live.

Climbing stairs, memoised

```
func climbStairs(_ n: Int) -> Int {
    var memo: [Int: Int] = [:]

    func ways(_ step: Int) -> Int {
        if step <= 2 { return max(step, 1) }         // base cases
        if let cached = memo[step] { return cached }

        let result = ways(step - 1) + ways(step - 2)  // recurrence
        memo[step] = result
        return result
    }

    return ways(n)
}
```

Without the cache this is $O(2^n)$. With it, every state is computed once, so it is $O(n)$.

Bottom-up: fill a table

Same recurrence, written as a loop. Usually faster and always free of stack-depth risk.

```
func climbStairs(_ n: Int) -> Int {
    guard n > 2 else { return max(n, 1) }
    var dp = Array(repeating: 0, count: n + 1)
    dp[1] = 1
    dp[2] = 2
    for i in 3...n {
        dp[i] = dp[i - 1] + dp[i - 2]
    }
    return dp[n]
}
```

Rolling the array away

When a state depends only on the last one or two, you do not need the table at all.

House robber, in O(1) space

```
func rob(_ nums: [Int]) -> Int {
    var skip = 0        // best if we do not take the previous house
    var take = 0        // best including the previous house

    for n in nums {
        let newTake = max(take, skip + n)
        skip = take
        take = newTake
    }
    return take
}
```

Offering this after the O(n)-space version is one of the cleanest optimisations you can show in an interview: same complexity in time, constant space, three lines shorter.

The 1-D template: unbounded choice

Coin change is the archetype. For each amount, try every coin and keep the best.

```swift
func coinChange(_ coins: [Int], _ amount: Int) -> Int {
    guard amount > 0 else { return 0 }
    let impossible = amount + 1                          // sentinel, safely too big
    var dp = Array(repeating: impossible, count: amount + 1)
    dp[0] = 0                                            // zero coins make zero

    for value in 1...amount {
        for coin in coins where coin <= value {
            dp[value] = min(dp[value], dp[value - coin] + 1)
        }
    }
    return dp[amount] == impossible ? -1 : dp[amount]
}
```

Cost O(amount × coins) time, O(amount) space. Note that this is pseudo-polynomial — it scales with the numeric value of the amount, not the size of the input. Say so if asked; it is a distinction interviewers like to probe.

The 2-D template: two sequences

When the state needs two indices — comparing two strings, or a grid — the table is two-dimensional and the recurrence looks at up, left, and diagonal.

Longest common subsequence

```swift
func longestCommonSubsequence(_ a: String, _ b: String) -> Int {
    let x = Array(a), y = Array(b)
    guard !x.isEmpty, !y.isEmpty else { return 0 }
    var dp = Array(repeating: Array(repeating: 0, count: y.count + 1),
                   count: x.count + 1)

    for i in 1...x.count {
        for j in 1...y.count {
            if x[i - 1] == y[j - 1] {
                dp[i][j] = dp[i - 1][j - 1] + 1          // match: extend the diagonal
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])   // skip one or the other
            }
        }
    }
    return dp[x.count][y.count]
}
```

The extra row and column of zeros is not decoration — it removes every bounds check from the recurrence. Sizing the table `n + 1` by `m + 1` is standard and worth doing by reflex.

Getting the state wrong is the real failure

If you find yourself needing information the state does not carry, the state is incomplete. When a problem involves a running constraint — a budget, a remaining count, whether the previous element was used — that constraint usually belongs *in* the state as another dimension. Adding it is normally the fix.

Recognising the sub-families

Family	State	Examples
Linear	index	climbing stairs, house robber, decode ways
Knapsack	index + remaining capacity	coin change, partition equal subset sum

Family	State	Examples
Two sequences	two indices	edit distance, LCS, regex matching
Grid	row + column	unique paths, minimum path sum
Interval	left + right	burst balloons, palindrome partitioning

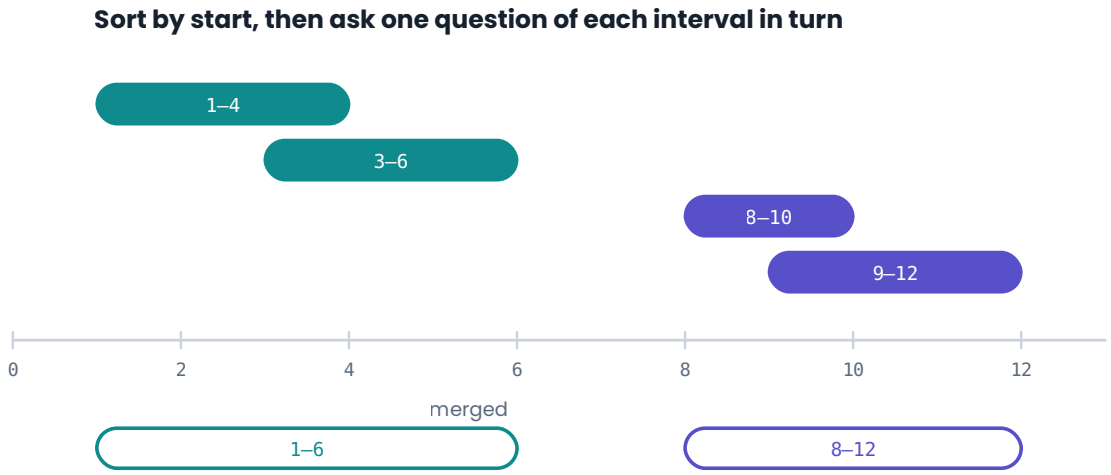
Drill these

Climbing stairs · House robber · Coin change · Longest increasing subsequence · Word break · Unique paths · Minimum path sum · Longest common subsequence · Edit distance · Partition equal subset sum

Chapter twenty-eight

Intervals

> **The signal** Pairs of start and end values — meetings, bookings, ranges. Merging, inserting, counting overlaps, or finding a free slot.



Sorting by start is what reduces the problem to a single comparison with the previous interval.

Sort first, nearly always

Unsorted, deciding whether two intervals overlap means comparing against all the others. Sorted by start, you only ever have to look at the one before, because anything earlier ended even sooner or was already absorbed.

> Two intervals `a` and `b` with `a.start ≤ b.start` overlap when **b.start ≤ a.end**. That single comparison is the whole pattern.

Merging

```swift
func merge(_ intervals: [[Int]]) -> [[Int]] {
    guard !intervals.isEmpty else { return [] }
    let sorted = intervals.sorted { $0[0] < $1[0] }
    var result: [[Int]] = [sorted[0]]

    for interval in sorted.dropFirst() {
        if interval[0] <= result[result.count - 1][1] {
            // overlap: stretch the last one to cover both
            result[result.count - 1][1] = max(result[result.count - 1][1],
                                              interval[1])
        } else {
            result.append(interval)
        }
    }
    return result
}
```

The `max` on the end is easy to drop and produces a subtly wrong answer only when one interval fully contains another — `[[1,10],[2,3]]` is the test that catches it.

Cost O(n log n) for the sort, O(n) for the pass. The sort dominates. O(n) space for the output.

Non-overlapping: sort by end instead

When you are choosing the largest set of intervals that do not clash — meeting rooms, activity selection — sorting by **end** is the greedy that works. Taking the earliest-finishing interval always leaves the most room for the rest.

Minimum removals to make the rest non-overlapping

```
func eraseOverlapIntervals(_ intervals: [[Int]]) -> Int {
    guard !intervals.isEmpty else { return 0 }
    let sorted = intervals.sorted { $0[1] < $1[1] }      // by end
    var kept = 1
    var lastEnd = sorted[0][1]

    for interval in sorted.dropFirst() where interval[0] >= lastEnd {
        kept += 1
        lastEnd = interval[1]
    }
    return intervals.count - kept
}
```

Why sorting by end is provably right

Among all intervals you could pick first, the one that finishes earliest leaves the largest remaining window. Any other choice leaves a window that is a subset of that one, so it cannot lead to a better answer. This exchange argument is short enough to say aloud, and saying it is much stronger than asserting "greedy works here".

The sweep line: counting concurrent intervals

When the question is "how many at once" — maximum meeting rooms, peak concurrency — forget the intervals and think about events. Every start is +1, every end is −1. Sort the events and walk them.

```
func minMeetingRooms(_ intervals: [[Int]]) -> Int {
    var events: [(time: Int, delta: Int)] = []
    for interval in intervals {
        events.append((interval[0], 1))
        events.append((interval[1], -1))
    }
    // ends before starts at the same instant: a room frees up in time
    events.sort { $0.time == $1.time ? $0.delta < $1.delta : $0.time < $1.time }

    var current = 0, peak = 0
    for event in events {
        current += event.delta
        peak = max(peak, current)
    }
    return peak
}
```

The tie-break is the entire difficulty. If a meeting ends at 10 and another starts at 10, they do not need two rooms — so the −1 must be processed first.

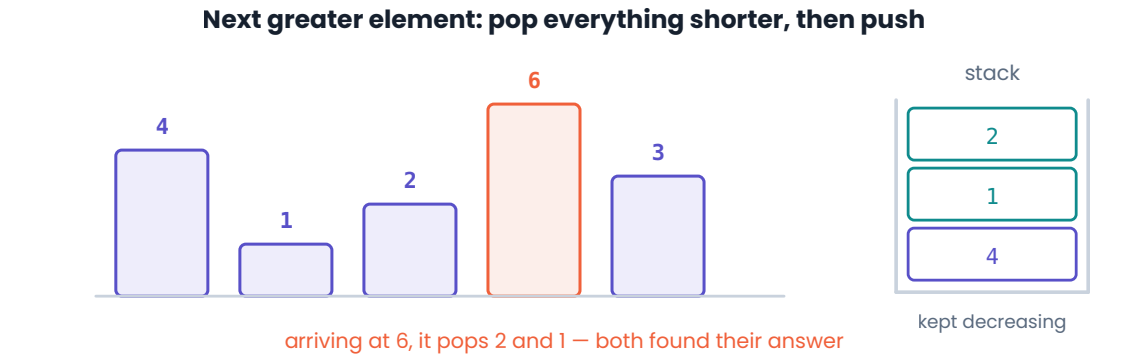
Drill these

Merge intervals · Insert interval · Non-overlapping intervals · Meeting rooms · Meeting rooms II · Interval list intersections · Car pooling

Chapter twenty-nine

Monotonic stack

The signal "The next greater", "the previous smaller", "how far until…", or anything about spans and histograms. The brute force is a nested loop scanning forward from each element.



Each index is pushed once and popped once — O(n), not O(n²).

The stack holds indices whose answer is still unknown.

The stack holds the indices still waiting for an answer, and it is always sorted.

The idea

Keep a stack whose values are always increasing (or always decreasing). Before pushing a new element, pop everything that breaks the order — and those popped elements have just found their answer, because the new element is the first one that beat them.

Every index is pushed once and popped once, so despite the inner `while`, the whole thing is O(n).

Template

Next greater element to the right

```
func nextGreater(_ nums: [Int]) -> [Int] {
    var result = Array(repeating: -1, count: nums.count)
    var stack: [Int] = []                        // indices, values decreasing

    for i in 0..<nums.count {
        while let top = stack.last, nums[top] < nums[i] {
            result[top] = nums[i]                // nums[i] is top's answer
            stack.removeLast()
        }
        stack.append(i)
    }
    return result                                // anything left keeps -1
}
```

Store **indices**, not values. You almost always need the distance between positions as well as the values, and an index gives you both.

Which direction, which order

```
next greater → pop while stack top is smaller → stack stays decreasing
next smaller → pop while stack top is larger → stack stays increasing
```

For "previous" instead of "next", either iterate right to left, or read the answer off the stack top at the moment you push rather than when you pop.

Worked example: daily temperatures

How many days until it gets warmer. Identical to the template, except the answer stored is a distance.

```swift
func dailyTemperatures(_ temperatures: [Int]) -> [Int] {
    var result = Array(repeating: 0, count: temperatures.count)
    var stack: [Int] = []

    for i in 0..<temperatures.count {
        while let top = stack.last, temperatures[top] < temperatures[i] {
            result[top] = i - top               // the distance, not the value
            stack.removeLast()
        }
        stack.append(i)
    }
    return result
}
```

Cost O(n) time — the inner loop runs at most n times in total across the whole outer loop, because each index is popped at most once. O(n) space. The brute force is O(n²).

Justify the linear bound out loud

A nested `while` inside a `for` looks quadratic, and an interviewer may well ask. The answer is amortised: n pushes total means at most n pops total, so the inner loop's whole lifetime is O(n). Being able to say that cleanly is most of the value of knowing this pattern.

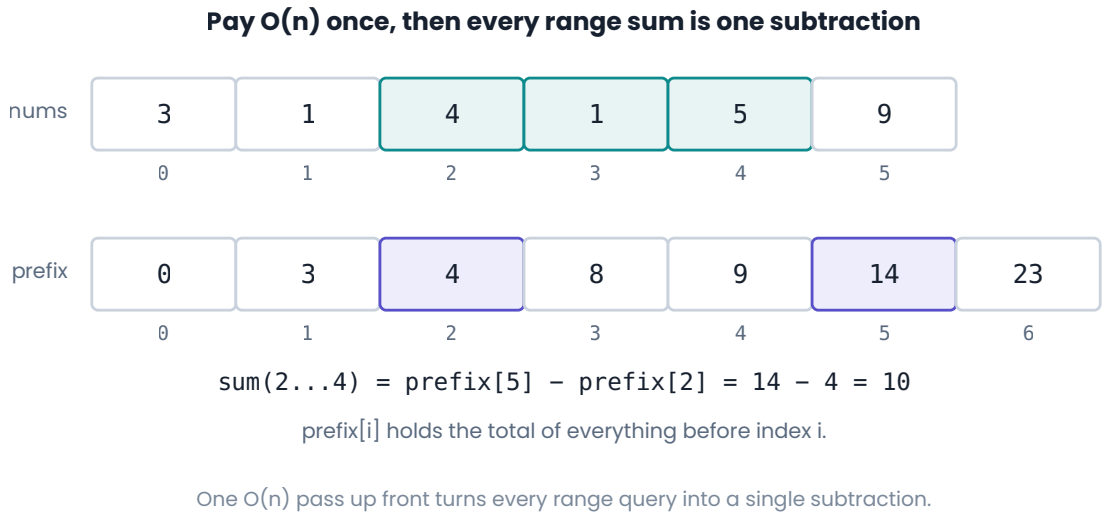
Drill these

Next greater element I and II · Daily temperatures · Largest rectangle in histogram · Trapping rain water · Remove k digits · Sum of subarray minimums

Chapter thirty

Prefix sums

> **The signal** Many queries about the sum of a range, or counting subarrays that sum to a target — especially when negative numbers rule out a sliding window.



One O(n) pass up front turns every range query into a single subtraction.

Building it

```swift
func buildPrefix(_ nums: [Int]) -> [Int] {
    var prefix = Array(repeating: 0, count: nums.count + 1)
    for i in 0..<nums.count {
        prefix[i + 1] = prefix[i] + nums[i]
    }
    return prefix
}

// sum of nums[left...right], inclusive
let rangeSum = prefix[right + 1] - prefix[left]
```

The leading zero, and sizing the array `n + 1`, is what makes that formula work for `left == 0` without a special case. Build it that way every time.

The real pattern: subarray sums with a dictionary

Range queries are the easy half. The version that appears in interviews counts subarrays summing to k, and it combines prefix sums with a hash map.

The insight: a subarray `(i, j]` sums to k exactly when `prefix[j] − prefix[i] == k`. Rearranged, `prefix[i] == prefix[j] − k`. So as you sweep, ask how many earlier prefixes had the value you need.

```swift
func subarraySum(_ nums: [Int], _ k: Int) -> Int {
    var countOfPrefix: [Int: Int] = [0: 1]     // the empty prefix has sum 0
    var running = 0
    var answer = 0

    for n in nums {
        running += n
        answer += countOfPrefix[running - k, default: 0]
        countOfPrefix[running, default: 0] += 1
    }
    return answer
}
```

Cost O(n) time, O(n) space. The brute force over every subarray is O(n²).

Seed the dictionary with [0: 1]

Without it you miss every subarray that starts at index 0, because no earlier prefix recorded the empty sum. It is a one-character bug that passes small tests and fails the moment the answer includes a prefix of the array.

Order matters

Look up `running - k` **before** recording `running`. If you record first, a k of zero lets a subarray match itself, and the count comes out too high.

Variations worth knowing

Prefix XOR — the same trick with `^` instead of `+`, since XOR is its own inverse. Counts subarrays with a given XOR.

Difference array — the inverse operation. To add a value across many ranges cheaply, record `+v` at the start and `-v` just past the end, then take the prefix sum once at the end. Range updates become O(1) each.

2-D prefix sums — for rectangle sums in a grid, with inclusion-exclusion:

```swift
// sum of the rectangle from (r1, c1) to (r2, c2), inclusive
let total = prefix[r2 + 1][c2 + 1]
          - prefix[r1][c2 + 1]
          - prefix[r2 + 1][c1]
          + prefix[r1][c1]        // added back: subtracted twice
```

Drill these

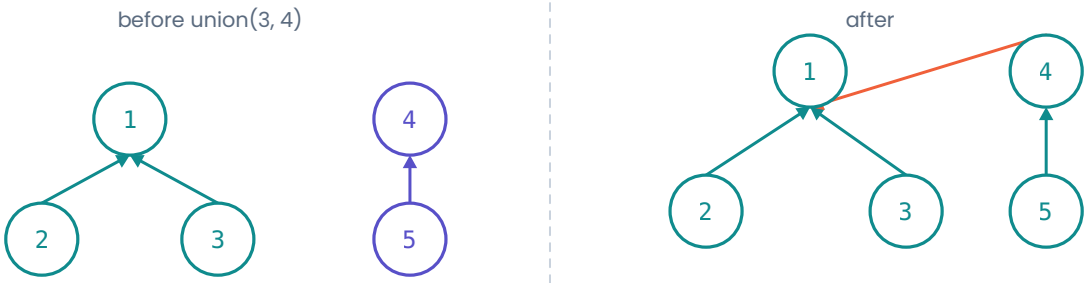
Range sum query — immutable · Subarray sum equals k · Contiguous array · Product of array except self · Find pivot index · Continuous subarray sum · Range sum query 2D

Chapter thirty-one

Union-Find

> **The signal** Connectivity that grows — "are these two connected", "how many groups", "does adding this edge create a cycle". Especially when edges arrive one at a time.

Union-Find answers one question: are these two in the same group?



Path compression flattens the tree as you search it, so lookups stay near O(1).

Each group is a tree, and the root is the group's name.

Why it exists

You could answer "are these connected" with a DFS every time, at O(V + E) per query. Union-Find answers it in near-constant time by keeping only what matters: for each element, a pointer towards its group's representative.

The implementation

Two optimisations turn it from mediocre to effectively O(1). **Path compression** flattens the tree during `find`. **Union by rank** attaches the shorter tree under the taller one so the trees never get deep.

```
struct UnionFind {
    private var parent: [Int]
    private var rank: [Int]
    private(set) var groupCount: Int

    init(_ n: Int) {
        parent = Array(0..<n)                    // everyone is their own root
        rank = Array(repeating: 0, count: n)
        groupCount = n
    }

    mutating func find(_ x: Int) -> Int {
        if parent[x] != x {
            parent[x] = find(parent[x])          // path compression
        }
        return parent[x]
    }

    @discardableResult
    mutating func union(_ a: Int, _ b: Int) -> Bool {
        let rootA = find(a), rootB = find(b)
        guard rootA != rootB else { return false }   // already together

        if rank[rootA] < rank[rootB] {
            parent[rootA] = rootB
        } else if rank[rootA] > rank[rootB] {
            parent[rootB] = rootA
        } else {
            parent[rootB] = rootA
            rank[rootA] += 1
        }
        groupCount -= 1
        return true
    }

    mutating func connected(_ a: Int, _ b: Int) -> Bool {
        find(a) == find(b)
    }
}
```

`union` returning `false` when the two were already joined is quietly useful: it is exactly the cycle test, and it is what makes Kruskal's minimum spanning tree a five-line algorithm.

Cost Both operations are O(α(n)) amortised, where α is the inverse Ackermann function. It is below 5 for any input that fits in a computer, so treating it as constant is fair — but say "effectively constant", not "constant".

Using it

Count connected components

```
func countComponents(_ n: Int, _ edges: [[Int]]) -> Int {
    var uf = UnionFind(n)
    for edge in edges {
        uf.union(edge[0], edge[1])
    }
    return uf.groupCount
}
```

Keeping the count on the struct and decrementing it inside `union` is neater than counting distinct roots afterwards, and it is O(1) to read.

Union-Find or DFS?

> Use **Union-Find** when edges arrive incrementally, when you are asked many connectivity questions, or when you need to detect a cycle as you build. Use **DFS** when the graph is fixed and you need the actual components, paths, or ordering — Union-Find tells you *whether* things are connected, never *how*.

Mapping a grid onto it

Union-Find indexes integers, so flatten two-dimensional coordinates first:

```
let id = row * cols + col            // and back: (id / cols, id % cols)
```

Drill these

Number of connected components · Graph valid tree · Number of islands (Union-Find version) · Redundant connection · Accounts merge · Most stones removed · Satisfiability of equality equations

Chapter thirty-two

Bit manipulation

> **The signal** "Without extra space", "every element appears twice except one", powers of two, or subsets encoded as a number. Uncommon, but unmistakable when it appears.

The operators

Operator	Meaning	Example
`&`	AND — 1 only where both are 1	`6 & 3 == 2`
`|`	OR — 1 where either is 1	`6 | 3 == 7`
`^`	XOR — 1 where they differ	`6 ^ 3 == 5`
`~`	NOT — flip every bit	`~0 == -1`
`<<`	shift left, doubling	`1 << 3 == 8`
`>>`	shift right, halving	`8 >> 2 == 2`

The four tricks worth memorising

```
n & 1 == 1                // odd?
n & (n - 1)               // clears the lowest set bit
n & (n - 1) == 0          // is n a power of two? (for n > 0)
n & -n                    // isolates the lowest set bit
```

`n & (n - 1)` is the one to understand rather than memorise. Subtracting one flips the lowest set bit to zero and everything below it to one; ANDing with the original keeps only the bits above. Counting how many times you can do that before reaching zero counts the set bits.

```
func countBits(_ n: Int) -> Int {
    var n = n, count = 0
    while n != 0 {
        n &= n - 1                        // remove one set bit per loop
        count += 1
    }
    return count
}
// or just: n.nonzeroBitCount
```

XOR, and why it solves "find the single one"

XOR has three properties that combine beautifully: `a ^ a == 0`, `a ^ 0 == a`, and it is commutative. So XOR everything together and all the pairs annihilate, leaving only the element that appeared once.

```
func singleNumber(_ nums: [Int]) -> Int {
    nums.reduce(0, ^)
}
```

O(n) time, O(1) space, one line. The hash-set solution is O(n) space, so this is a genuine improvement rather than a party trick.

Subsets as bitmasks

With n ≤ 20 or so, every subset of a set can be represented by an integer whose bits say which elements are in. Counting from 0 to $2^n - 1$ enumerates them all.

```
func subsets(_ nums: [Int]) -> [[Int]] {
    var result: [[Int]] = []

    for mask in 0..<(1 << nums.count) {
        var subset: [Int] = []
        for i in 0..<nums.count where mask & (1 << i) != 0 {
            subset.append(nums[i])
        }
        result.append(subset)
    }
    return result
}
```

This is the same output as the backtracking version in chapter 24, without recursion. It is also the foundation of bitmask dynamic programming, where the state is "which items have I used".

Swift specifics

```
Int.bitWidth                // 64
n.nonzeroBitCount           // population count
n.leadingZeroBitCount
n.trailingZeroBitCount
UInt8(truncatingIfNeeded: n)
n &+ 1                      // wrapping add — no overflow trap
```

Right-shifting a negative number

Swift's `>>` on a signed `Int` is an arithmetic shift: it keeps the sign bit, so `-8 >> 1` is `-4`, and shifting a negative number right can never reach zero. If a problem needs a logical shift, convert to `UInt` first. This is the bug that makes bit-counting loops hang on negative input.

Drill these

Single number · Single number II · Number of 1 bits · Counting bits · Reverse bits · Missing number · Sum of two integers · Subsets (bitmask version)

Part four

The iOS engineer

Algorithms get you through one round. These chapters are the rest of the loop — memory, concurrency, the frameworks, and the questions about how you would actually build the thing.

Chapter thirty-three

Memory and ARC

The most reliably asked non-algorithmic question in an iOS interview. Not because leaks are exotic, but because the answer reveals in about ninety seconds whether you understand object lifetimes.

What ARC actually does

Automatic Reference Counting is not garbage collection. There is no runtime scanning for unreachable objects. Instead the **compiler** inserts retain and release calls at compile time, and an object is deallocated the instant its strong reference count hits zero.

Two consequences follow, and both come up in interviews. Deallocation is deterministic — you know exactly when `deinit` runs. And a cycle is never collected, because nothing is looking for one.

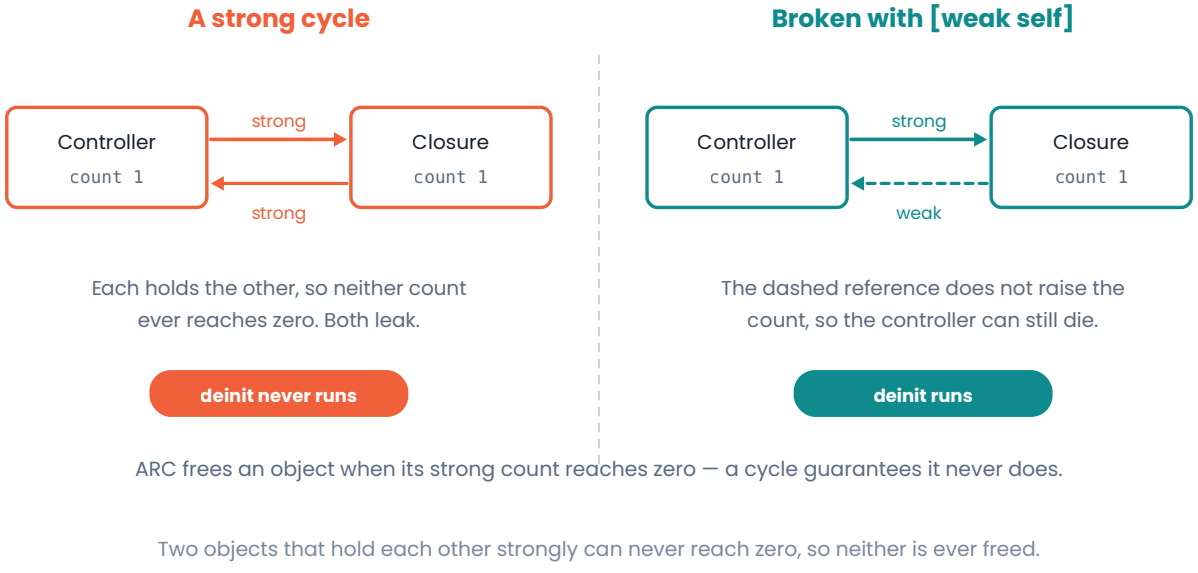
> ARC applies to **reference types** only. Structs, enums and tuples are copied, not counted — though a struct holding a class property still participates through that property.

The three reference kinds

```
strong    the default; keeps the object alive
weak      does not keep it alive; must be var and Optional; nils itself when the object goes
unowned   does not keep it alive; non-Optional; a crash if accessed after deallocation
```

The choice between `weak` and `unowned` is a claim about lifetimes. Use `unowned` only when the referenced object is guaranteed to outlive the reference. When in doubt, `weak` — a nil check is cheaper than a crash report.

Retain cycles



ARC frees an object when its strong count reaches zero — a cycle guarantees it never does.

Two objects that hold each other strongly can never reach zero, so neither is ever freed.

Three situations produce nearly all real cycles:

Parent and child holding each other. The fix is that the child's reference back to the parent is `weak`. This is why delegate properties are almost always declared `weak var delegate: SomeDelegate?`.

A closure stored as a property that captures `self`. The object owns the closure, and the closure captures the object.

A repeating `Timer` or notification observer that captures `self` and is never invalidated.

The cycle

```
final class ImageLoader {
    var onComplete: ((UIImage) -> Void)?

    func load() {
        onComplete = { image in
            self.cache(image)      // the closure now owns self, and self owns it
        }
    }
}
```

The fix — a capture list

```
onComplete = { [weak self] image in
    guard let self else { return }     // becomes a strong local for this scope
    self.cache(image)
}
```

`[weak self]` makes `self` Optional inside the closure. The `guard let self` line promotes it back to a non-Optional for the duration of the block, so it cannot be deallocated halfway through your code.

> **Not every closure needs a capture list**
>
> A closure that is executed and discarded — the body of `map`, a `UIView.animate` block, a one-shot `URLSession` completion — does not create a cycle, because nothing holds it afterwards. Adding `[weak self]` everywhere is cargo cult, and an interviewer may well ask you to justify it. The question to answer is: *does the object own this closure?*

Delegates

```
protocol DataSourceDelegate: AnyObject {     // AnyObject is required for weak
    func didUpdate()
}

final class DataSource {
    weak var delegate: DataSourceDelegate?
}
```

The `AnyObject` constraint restricts the protocol to class types. Without it the protocol could be adopted by a struct, and `weak` on a value type is meaningless — so the compiler rejects it.

Proving it in an interview

```
deinit {
    print("\(type(of: self)) deallocated")
}
```

A `deinit` that never prints is a leak. Saying "I'd add a `deinit` and check it fires, then confirm with the Memory Graph Debugger and the Leaks instrument" is a complete, practical answer.

> **The follow-up you should expect**
>
> "When would you use `unowned` instead of `weak`?" The strongest answer is concrete: when the closure and the object have identical lifetimes and the reference can never outlive the object — for instance a lazily initialised property that refers back to its own owner. Then add that you would still reach for `weak` under uncertainty, because the failure mode is nil rather than a crash.

Value types change the question

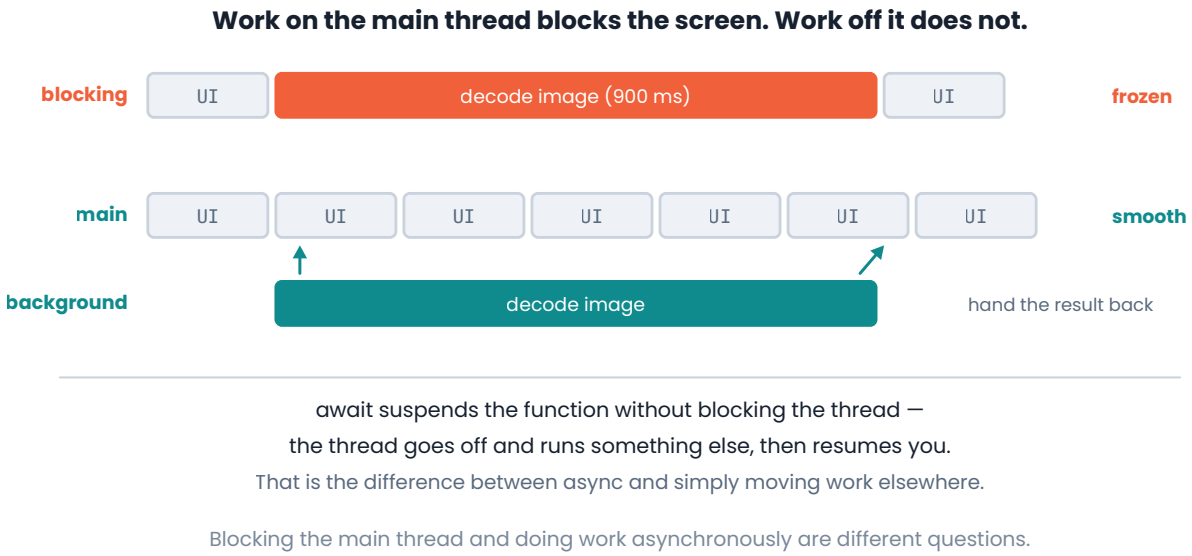
Because structs are copied, they cannot participate in a reference cycle at all. That is one of the real arguments for value types beyond thread safety, and it is worth saying out loud: choosing a struct removes an entire class of bug rather than defending against it.

The cost is copying. Swift softens that with copy-on-write for the standard library collections, but a large struct passed around and mutated repeatedly does real work. As always, measure before optimising.

Chapter thirty-four

Concurrency

The framework story changed substantially with async/await, and interviews now expect fluency in both the modern model and the GCD code you will still find in every real codebase.



The one rule

UIKit and SwiftUI must be touched from the main thread only. Everything else in iOS concurrency is a technique for honouring that rule while still doing slow work.

The main thread is also the thread that renders. Anything you run there competes with drawing the next frame, which is why a slow function on the main thread appears to the user as a frozen screen rather than a slow feature.

async/await

`await` marks a **suspension point**. The function pauses, the thread is released to do other work, and the function resumes later — possibly on a different thread.

```
func loadProfile(id: String) async throws -> Profile {
    let (data, response) = try await URLSession.shared.data(from: url(for: id))
    guard let http = response as? HTTPURLResponse, http.statusCode == 200 else {
        throw LoadError.badResponse
    }
    return try JSONDecoder().decode(Profile.self, from: data)
}
```

That reads top to bottom, and errors propagate with ordinary `throws` — which is the entire argument for it over completion handlers.

Structured concurrency

Child tasks are bound to the scope that created them. When the scope exits, they are cancelled. That is what "structured" means, and it is why leaked background work is much harder to write in this model.

Two requests concurrently, not one after the other

```
async let profile = loadProfile(id: id)
async let posts = loadPosts(id: id)
let (loadedProfile, loadedPosts) = try await (profile, posts)
```

A dynamic number of concurrent children

```
let images = try await withThrowingTaskGroup(of: UIImage.self) { group in
    for url in urls {
        group.addTask { try await loadImage(from: url) }
    }
    var result: [UIImage] = []
    for try await image in group {
        result.append(image)      // completion order, not submission order
    }
    return result
}
```

Task group results arrive out of order

Iterating a group yields results as they finish, not in the order you added them. If order matters, add the index to the child's return type and sort afterwards, or write into a pre-sized array by index. Forgetting this is one of the most common bugs in real async code.

Actors

An actor protects its own mutable state. Only one task touches an actor's state at a time, so data races inside it are impossible by construction.

```
actor ImageCache {
    private var storage: [URL: UIImage] = [:]

    func image(for url: URL) -> UIImage? { storage[url] }

    func store(_ image: UIImage, for url: URL) {
        storage[url] = image
    }
}

let cached = await cache.image(for: url)     // await, because it is isolated
```

Calls from outside must `await`, because the caller may have to wait its turn. That `await` is the visible cost of the safety.

`@MainActor` is a global actor that represents the main thread. Marking a type or method with it is the modern replacement for `DispatchQueue.main.async`:

```
@MainActor
final class ProfileViewModel {
    var state: ViewState = .loading      // now only mutated on the main thread
}
```

Actor reentrancy

An actor guarantees that only one task runs its code at a time — it does **not** guarantee that a method runs start to finish without interruption. At every `await` inside an actor method, another task may enter. So state you read before an `await` may have changed after it. Re-check your assumptions after every suspension point. This is a favourite senior-level question and very few candidates get it right.

GCD, which you still need to know

```
DispatchQueue.global(qos: .userInitiated).async {
    let result = expensiveWork()
    DispatchQueue.main.async {
        self.label.text = result       // back to main for UI
    }
}
```

Serial queues run one block at a time and are a lightweight lock. Concurrent queues run many. The quality-of-service levels — `.userInteractive`, `.userInitiated`, `.utility`, `.background` — tell the system how to prioritise, and choosing them thoughtfully is a real signal.

Never call DispatchQueue.main.sync from the main thread

It deadlocks immediately: the main queue waits for a block that cannot start until the main queue is free. This is a classic interview question and the answer is one sentence — the queue is blocked waiting on itself.

Bridging old to new

```
func loadImage(from url: URL) async throws -> UIImage {
    try await withCheckedThrowingContinuation { continuation in
        legacyLoader.load(url) { result in
            switch result {
            case .success(let image): continuation.resume(returning: image)
            case .failure(let error): continuation.resume(throwing: error)
            }
        }
    }
}
```

A continuation must be resumed **exactly once**. Resuming twice is a crash; never resuming leaks the task forever. The checked variants exist precisely to catch both mistakes during development.

Sendable and data races

`Sendable` marks a type as safe to pass across concurrency boundaries. Value types of `Sendable` members get it automatically; classes need `final` plus immutable state, or manual synchronisation. Under Swift 6 language mode these checks become errors rather than warnings, and knowing that the direction of travel is compile-time data-race safety is worth mentioning.

Chapter thirty-five

UIKit

SwiftUI gets the attention, but the large apps are still substantially UIKit, and interviewers ask about it because the lifecycle questions have precise answers.

View controller lifecycle

Method	When	What belongs there
`viewDidLoad`	once, after the view is created	one-time setup, subview creation
`viewWillAppear`	before every appearance	refreshing data that may be stale
`viewDidAppear`	after every appearance	starting animations, analytics
`viewWillDisappear`	before leaving	saving state, resigning first responder
`viewDidDisappear`	after leaving	stopping timers and observers

The distinction that matters: `viewDidLoad` runs **once**, the appear methods run **every time**. Putting an observer registration in `viewDidAppear` without removing it in `viewDidDisappear` registers it repeatedly, which is a common real-world bug and a fair interview question.

Frames are not final in viewDidLoad

The view exists but has not been laid out, so its bounds are whatever the storyboard or nib said, not what the device will use. Anything depending on final size belongs in `viewDidLayoutSubviews` — which may run several times, so keep it idempotent and cheap.

The layout cycle

Layout is deferred and batched. You mark something as needing work, and the system does it before the next frame is drawn.

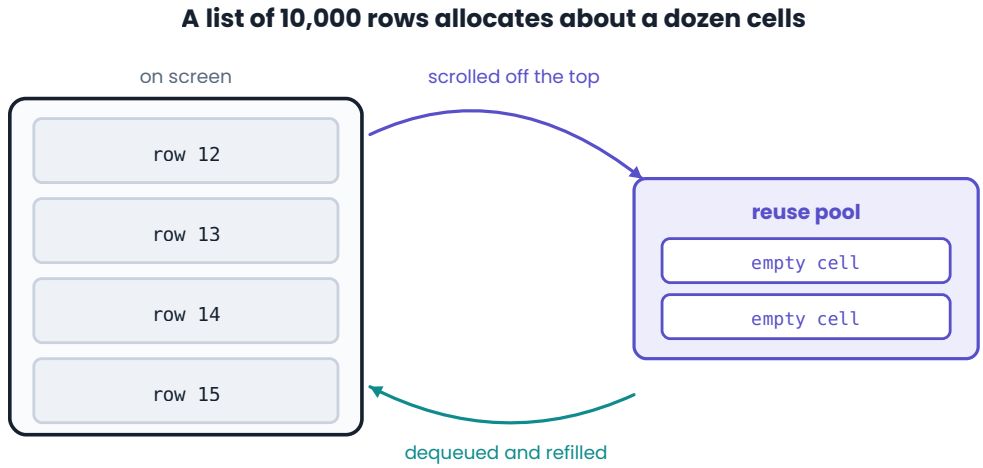
```
setNeedsLayout    mark dirty, return immediately — layout happens later
layoutIfNeeded    force it to happen now, synchronously
layoutSubviews    the method the system calls; override it, never call it
```

Auto Layout solves a system of constraints. Each constraint has a priority from 1 to 1000, and `required` is 1000. The two properties that decide what happens when there is not enough room are **content hugging** (resistance to growing) and **compression resistance** (resistance to shrinking) — being able to explain those two by name is a genuinely differentiating answer.

Constraints in code

```
view.addSubview(label)
label.translatesAutoresizingMaskIntoConstraints = false   // forget this and nothing works
NSLayoutConstraint.activate([
    label.leadingAnchor.constraint(equalTo: view.leadingAnchor, constant: 16),
    label.trailingAnchor.constraint(equalTo: view.trailingAnchor, constant: -16),
    label.topAnchor.constraint(equalTo: view.safeAreaLayoutGuide.topAnchor)
])
```

Cell reuse



Which is why prepareForReuse exists: the cell handed back to you still holds the last row's contents.

Scrolling recycles a small pool of cells rather than creating one per row.

```
func tableView(_ tableView: UITableView,
               cellForRowAt indexPath: IndexPath) -> UITableViewCell {
    let cell = tableView.dequeueReusableCell(withIdentifier: "Cell",
                                             for: indexPath) as! MyCell
    cell.configure(with: items[indexPath.row])
    return cell
}
```

The cell you receive is usually one that was just on screen, still holding the previous row's content. Two rules follow: configure **every** property every time — never only the ones that changed — and clear anything asynchronous in `prepareForReuse()`.

```
override func prepareForReuse() {
    super.prepareForReuse()
    imageView.image = nil
    imageLoadTask?.cancel()      // otherwise a slow download lands on the wrong row
}
```

That last line is the classic bug: scroll fast, and images appear briefly in the wrong cells. If an interviewer describes that symptom, the answer is cancelling the in-flight request on reuse and verifying the response still matches the row before applying it.

Diffable data sources

The modern replacement for `reloadData`. You describe the new state as a snapshot and the framework computes the animated difference.

```
var snapshot = NSDiffableDataSourceSnapshot<Section, Item>()
snapshot.appendSections([.main])
snapshot.appendItems(items)
dataSource.apply(snapshot, animatingDifferences: true)
```

It requires `Item` to be `Hashable`, and the identity must be stable — using an index or a mutable field as the hash produces animations that flicker or crash. Knowing why identity matters here transfers directly to SwiftUI's `id`.

The responder chain

Events travel up a chain: from the first responder, through the view hierarchy, to the view controller, then the window, then the application. It is how a button tap deep in a hierarchy can be handled far above it, and how `becomeFirstResponder` and keyboard handling work.

> **A word on Objective-C**
>
> Apple maintains very large Objective-C codebases and interoperability is routine. You do not need to write it fluently, but you should be able to read a header, know that `nil`-messaging is safe (sending a message to `nil` is a no-op rather than a crash), and know that `@objc` and `dynamic` expose Swift to the Objective-C runtime for things like selectors and KVO.

Chapter thirty-six

SwiftUI

The mental shift is that a view is a value describing what should be on screen, not an object you mutate. Almost every SwiftUI bug and interview question comes back to that one idea.

A SwiftUI view is a description, not the thing on screen

You never tell SwiftUI what to change. You describe what it should look like now, and it works out the smallest edit from the previous description.

This is why an expensive computation inside body is a performance bug: body is called far more often than you expect.

body is re-run and the result diffed. It is not a list of instructions to change something.

State and its wrappers

Wrapper	Who owns the value	Use for
`@State`	this view	private, view-local value state
`@Binding`	someone else	a read-write reference to a parent's state
`@Observable`	a reference type	model objects shared across views
`@Environment`	the environment	dependencies passed implicitly down the tree

```
struct CounterView: View {
    @State private var count = 0                // owned here

    var body: some View {
        VStack {
            Text("\(count)")
            StepperRow(value: $count)           // $ makes a Binding
        }
    }
}

struct StepperRow: View {
    @Binding var value: Int                     // owned by the parent
    var body: some View {
        Button("Add") { value += 1 }
    }
}
```

`@State` should be `private`. It is storage that belongs to this view, and letting a parent set it defeats the ownership model — the compiler will not stop you, but reviewers will.

For reference-type models, the `@Observable` macro (iOS 17 and later) replaced `ObservableObject` and `@Published`. It tracks which properties a view actually reads, so a

change to an unread property does not invalidate that view — a meaningful performance improvement over the older model, and a good thing to be able to explain.

```
@Observable
final class FeedModel {
    var state: ViewState = .loading

    func load() async { ... }
}
```

Identity

SwiftUI decides whether two views across two renders are "the same view" — which determines whether state is preserved or thrown away, and whether a transition animates or snaps.

Structural identity comes from position in the view tree. **Explicit identity** comes from `id:` in a `ForEach` or an `.id()` modifier.

```
ForEach(items) { item in ... }        // needs Identifiable, with a stable id
```

> **Never use the array index as an identity**
>
> `ForEach(0..<items.count)` or an id derived from position means that deleting the first element renumbers everything after it. SwiftUI concludes that every row changed, so state moves to the wrong rows and animations are wrong. Use a stable identifier from the model — the same requirement as a diffable data source.

Why body must be cheap

`body` is a computed property and it runs far more often than you would guess — any time a dependency changes anywhere up the tree. Sorting an array, formatting dates, or building a `DateFormatter` inside `body` is a performance bug. Compute in the model, or cache it.

```
// wrong: allocated on every single render
var body: some View {
    Text(DateFormatter().string(from: date))
}
```

Common interview questions

"When would you still choose UIKit?" Highly custom collection layouts, fine-grained scroll or text control, deep integration with existing UIKit code, or supporting an older deployment target. Note that mixing is normal: `UIHostingController` embeds SwiftUI in UIKit, and `UIViewRepresentable` goes the other way.

"How does SwiftUI decide what to redraw?" It re-runs `body`, compares the new description with the old, and applies the minimal set of changes. Views are cheap value types precisely so this comparison is cheap.

"`@State` versus `@Observable`?" `@State` for value state owned by one view; `@Observable` for a reference-type model shared across several. If two views must see the same mutation, it is a model.

Chapter thirty-seven

Networking and Codable

Every app is a client for something. This is the chapter most likely to turn into a practical exercise, because it is what the job is.

URLSession

```
func fetch<T: Decodable>(_ type: T.Type, from url: URL) async throws -> T {
    var request = URLRequest(url: url)
    request.httpMethod = "GET"
    request.setValue("application/json", forHTTPHeaderField: "Accept")

    let (data, response) = try await URLSession.shared.data(for: request)

    guard let http = response as? HTTPURLResponse else {
        throw NetworkError.invalidResponse
    }
    guard 200..<300 ~= http.statusCode else {
        throw NetworkError.status(http.statusCode)
    }

    let decoder = JSONDecoder()
    decoder.keyDecodingStrategy = .convertFromSnakeCase
    decoder.dateDecodingStrategy = .iso8601
    return try decoder.decode(T.self, from: data)
}
```

Three details separate this from tutorial code: a real status-code check, a typed error rather than a bare `throw`, and the decoder configured once rather than relying on the JSON matching your property names by luck.

> **A non-2xx response is not an error to URLSession**
>
> A 404 or a 500 arrives as a perfectly successful response with a body. `URLSession` only throws for transport failures — no network, DNS failure, timeout, cancellation. You must check the status code yourself, and forgetting to is one of the most common bugs in shipping apps.

Codable

```
struct Movie: Decodable {
    let id: String
    let title: String
    let releaseDate: Date?
    let runtimeMinutes: Int?

    enum CodingKeys: String, CodingKey {
        case id
        case title = "name"              // when the API disagrees with you
        case releaseDate = "release_date"
        case runtimeMinutes = "runtime"
    }
}
```

Make a field Optional when the server may genuinely omit it. Do **not** make everything Optional to avoid decoding failures — that pushes the problem into every call site. A decoding error on a field you require is information, and it should be surfaced rather than hidden.

For genuinely messy APIs, a custom `init(from decoder:)` lets you handle a field that arrives as a string on Tuesdays and a number on Wednesdays:

```
init(from decoder: Decoder) throws {
    let container = try decoder.container(keyedBy: CodingKeys.self)
    id = try container.decode(String.self, forKey: .id)
    if let number = try? container.decode(Int.self, forKey: .runtimeMinutes) {
        runtimeMinutes = number
    } else if let text = try? container.decode(String.self, forKey: .runtimeMinutes) {
        runtimeMinutes = Int(text)
    } else {
        runtimeMinutes = nil
    }
    ...
}
```

Errors worth modelling

```
enum NetworkError: Error {
    case invalidResponse
    case status(Int)
    case decoding(Error)
    case offline
}
```

Distinguishing these matters because the app's response differs: a 401 means re-authenticate, a 500 means offer a retry, offline means show cached content, and a decoding failure means the app is broken and should be logged. Collapsing them all into "something went wrong" is the design decision an interviewer will push on.

Cancellation

```
let task = Task {
    let profile = try await fetch(Profile.self, from: url)
    await MainActor.run { self.profile = profile }
}
// later, when the view goes away
task.cancel()
```

Cancellation is cooperative — a task is not killed, it is *asked* to stop, and it must check. `URLSession`'s async methods check for you and throw `CancellationError`. In your own long loops, call `try Task.checkCancellation()`.

Things worth mentioning unprompted

Caching. `URLCache` handles HTTP caching for free when the server sends the right headers. `ETag` and `If-None-Match` let the server reply 304 with no body.

Retries. Retry idempotent requests only, with exponential backoff and jitter, and cap the attempts.

Pagination. Cursor-based beats offset-based, because offsets shift when items are inserted while the user is paging.

Security. HTTPS via App Transport Security by default; certificate pinning for high-value APIs; never put tokens in `UserDefaults` — that is what the Keychain is for.

Chapter thirty-eight

Persistence

Choosing where data lives is a design question with a small number of right answers, and interviewers ask it because the wrong choice is a security incident rather than a bug.

The options, and when each is correct

Store	Good for	Never for
`UserDefaults`	small preferences, flags, last-selected tab	secrets, large blobs, structured data
Keychain	tokens, passwords, keys	anything large or frequently read
Files	images, documents, downloaded media, JSON caches	data you must query or relate
SQLite	complex queries, large datasets, full control	trivial key-value needs
Core Data	object graphs with relationships, undo, syncing	simple flat lists
SwiftData	the same, with a modern Swift API (iOS 17+)	apps supporting older systems

UserDefaults is a plist, and it is not secure

It is stored unencrypted inside the app container. Any access token placed there is readable on a jailbroken device or from an unencrypted backup. Tokens, passwords and keys belong in the Keychain, which is hardware-backed and survives app reinstall. This is a question with exactly one correct answer, and getting it wrong is memorable in the wrong way.

Files, and the directory that matters

```
let documents = FileManager.default.urls(for: .documentDirectory,
                                         in: .userDomainMask)[0]
let url = documents.appendingPathComponent("feed.json")
try JSONEncoder().encode(items).write(to: url, options: .atomic)
```

`.atomic` writes to a temporary file and renames it, so a crash mid-write cannot leave a truncated file behind. It is the right default for anything you will read later.

The distinction to know: **Documents** is user-generated data and is backed up; **Caches** is regenerable data and the system may delete it under storage pressure; **tmp** may be cleared at any time. Putting a large regenerable download in Documents will get an app rejected for backing up data it should not.

Core Data in one page

An **NSManagedObjectContext** is a scratchpad; changes exist only in memory until saved. The **persistent container** owns the stack. The rule that causes most bugs: a context and its managed objects are bound to one thread or queue. Use `perform` to touch a context safely, keep a main-queue context for the UI, and use background contexts for imports.

```
container.performBackgroundTask { context in
    // heavy import work off the main queue
    try? context.save()
}
```

`NSFetchedResultsController` is the classic pairing with a table view: it watches a fetch request and reports fine-grained changes, so the list animates rather than reloading.

SwiftData is the modern layer over the same machinery — `@Model`, `@Query`, and a `ModelContainer` — with far less ceremony. If asked to choose today for an iOS 17+ target, SwiftData is a defensible answer; if the app supports older systems, Core Data is still the one.

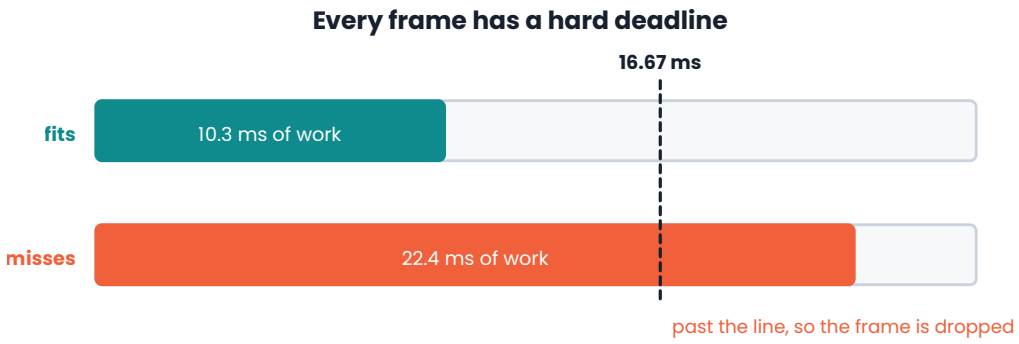
Migration

The moment your model changes, existing users have data in the old shape. Lightweight migration handles additive changes automatically. Anything else needs a mapping model. Mentioning migration unprompted signals that you have shipped an app rather than built one.

Chapter thirty-nine

Performance

Performance work on iOS is mostly one discipline: keep the main thread free, and measure before you believe anything.



60 Hz gives you 16.67 ms per frame. A 120 Hz ProMotion display gives you 8.33 ms. The user does not experience a slow function. They experience a stutter.

This is why anything slow belongs off the main thread.

The budget is fixed. Everything you do on the main thread comes out of it.

The frame budget

At 60 Hz you have 16.67 ms to produce a frame; on a 120 Hz ProMotion display, 8.33 ms. Exceed it and a frame is dropped — a visible stutter. This reframes performance usefully: the question is never "is this fast" but "does this fit in the budget, on the main thread, on the oldest device we support".

Where the time actually goes

Image decoding. A JPEG is decompressed to a bitmap the first time it is drawn — on the main thread, if you are careless. Decode off-thread and downsample to the size you will actually display. A 4000-pixel image in a 100-point thumbnail wastes both time and a great deal of memory.

Layout. Deeply nested Auto Layout hierarchies solve a bigger constraint system every pass. Flatten where you can, and avoid forcing synchronous `layoutIfNeeded` in a scroll loop.

Blocking I/O. Disk reads, `Data(contentsOf:)`, JSON decoding, and Core Data fetches on the main thread all directly consume the budget.

Off-screen rendering. Shadows without a `shadowPath`, and some masking and rounded-corner combinations, force an extra rendering pass.

Over-invalidation. In SwiftUI, work in `body` or state placed too high in the tree causes re-renders far larger than needed.

Instruments, by symptom

```
Time Profiler        what is consuming CPU, and on which thread
Allocations          memory growth and what is holding it
Leaks                objects never freed — pair with the Memory Graph Debugger for cycles
Animation Hitches    dropped frames and the work that caused them
Network              request timing, sizes and redundancy
```

The answer an interviewer wants to "the app is slow, what do you do" is a **process**, not a guess: reproduce it, measure with the right instrument, find the specific expensive thing, fix that one thing, and measure again to confirm. Volunteering an optimisation before measuring is the wrong-shaped answer even when the optimisation is correct.

Launch time

Cold launch is a metric Apple cares about a great deal. Keep `application(_:didFinishLaunchingWithOptions:)` minimal, defer non-essential setup, avoid synchronous network calls and disk reads on the launch path, and lazily initialise anything the first screen does not need.

Memory

`didReceiveMemoryWarning` and the equivalent notification are your cue to drop caches. `NSCache` is preferable to a plain dictionary for caching because it evicts automatically under pressure and is thread-safe. Large images are the usual cause of a memory termination, and downsampling is usually the fix.

> **The strongest thing you can say**
>
> "I would not guess — I would profile it." Then name the instrument you would open and the metric you would look at. Interviewers are checking for evidence-driven habits, and this one sentence demonstrates them more convincingly than a list of micro-optimisations.

Chapter forty

Testing

Testability is an architecture property, not a testing one. Most "how would you test this" questions are really asking whether you can find the seam.

The seam is dependency injection

Untestable code reaches out and grabs its dependencies. Testable code is handed them.

Untestable

```
final class ProfileLoader {
    func load() async throws -> Profile {
        try await URLSession.shared.data(from: url)   // hard-wired to the network
        ...
    }
}
```

Testable

```
protocol HTTPClient {
    func data(from url: URL) async throws -> (Data, URLResponse)
}

extension URLSession: HTTPClient {}

final class ProfileLoader {
    private let client: HTTPClient

    init(client: HTTPClient = URLSession.shared) {   // default keeps call sites simple
        self.client = client
    }
}
```

The default parameter is the detail worth copying: production code is unchanged, and tests pass a stub. That one change is usually the whole answer to "how would you test this".

Writing the test

```
final class ProfileLoaderTests: XCTestCase {

    func testDecodesProfileFromValidJSON() async throws {
        let client = StubClient(data: validJSON, statusCode: 200)
        let loader = ProfileLoader(client: client)

        let profile = try await loader.load()

        XCTAssertEqual(profile.name, "Ada")
    }

    func testThrowsOnServerError() async {
        let client = StubClient(data: Data(), statusCode: 500)
        let loader = ProfileLoader(client: client)

        do {
            _ = try await loader.load()
            XCTFail("expected an error")
        } catch NetworkError.status(let code) {
            XCTAssertEqual(code, 500)
        } catch {
            XCTFail("wrong error: \(error)")
        }
    }
}
```

Note the test names. `testDecodesProfileFromValidJSON` says what the behaviour is; `testLoad` says nothing. A reviewer reading a failure list should learn what broke without opening the file.

Modern Swift also has the Swift Testing framework, with `@Test` and `#expect` in place of `XCTAssert`. Mentioning that you know both, and that XCTest remains everywhere in existing codebases, is the accurate position.

What to test, and what not to

> Test **logic you wrote**: parsing, state transitions, business rules, edge cases. Do not test **the framework** — that a `UILabel` displays text is Apple's test, not yours. If something is hard to test, that is usually a design signal rather than a testing problem.

The pyramid still holds: many fast unit tests, fewer integration tests, a small number of UI tests covering critical journeys. UI tests are slow and flaky in proportion to how many you have, so spend them carefully — sign-in, checkout, the one flow that must never break.

Async and asynchrony

Modern async tests are simply `async throws` test methods with `await` in the body. For callback-based code that you cannot change, `XCTestExpectation` with `wait(for:timeout:)` remains the tool.

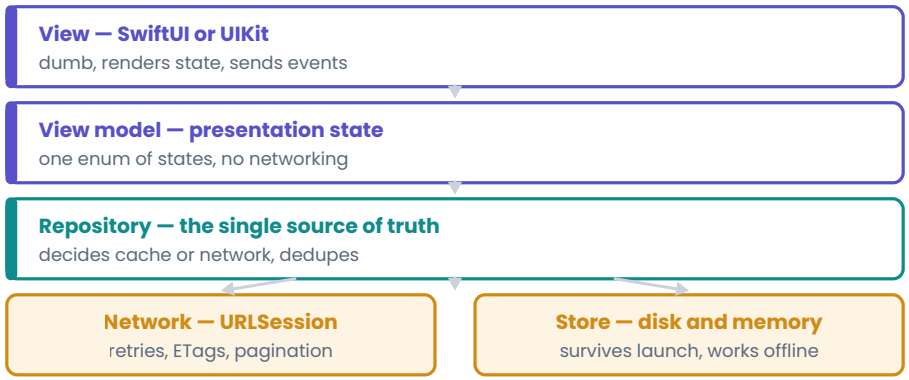
> **A question you should ask them**
>
> "What does your test suite look like, and do you trust it?" It is a real question with an informative answer, and it signals that you think of tests as infrastructure rather than a box to tick.

Chapter forty-one

The system design round

Not distributed systems. iOS system design is about the app: its layers, its data flow, its failure states, and the tradeoffs you chose deliberately.

The shape most iOS system design answers should take



Each layer is replaceable by a fake, which is the same reason it is testable.
Name the boundaries out loud — that is most of what the round is scoring.

Naming the layers and their responsibilities is most of what this round measures.

How to run the forty-five minutes

Clarify first, for five minutes. What exactly are we building? Which platforms and OS versions? Does it work offline? How large is the data? Who is the user? Is there an existing API or do we design it? Resist the urge to start drawing.

Then state the scope you are committing to. "I'll design the feed screen, its caching, and the image pipeline, and I'll skip authentication unless you want it." This makes the rest of the conversation legible.

Then work outside in: the screens and their states, the data model, the API contract, the storage and caching, the threading, and finally the failure and edge cases.

The layers to name

Almost any answer benefits from the same separation: a **view** that only renders state and reports events; a **view model** that holds presentation state as a single enum; a **repository** that decides between cache and network and is the single source of truth; and beneath it the **network** and **store** services.

The reason to be explicit about this is not architectural purity. It is that each boundary is a place where a fake can be substituted, which makes the whole thing testable — and saying that out loud connects the design to the previous chapter.

State, modelled honestly

```
enum ViewState {
    case loading
    case loaded([Item])
    case empty
    case failed(Error)
}
```

Interviewers notice when a candidate handles the empty case and the error case as first-class states rather than afterthoughts. Real apps spend a lot of their life in those states, and designing them is part of the job.

The classic prompts

Design an image loading library. The expected components: an in-memory cache (`NSCache`, keyed by URL), a disk cache with an eviction policy, deduplication so ten cells requesting the same URL make one request, cancellation on cell reuse, downsampling to the display size, and decoding off the main thread. Nearly every part of this book's iOS half shows up in that one question.

Design an offline-first feed. Local store as the source of truth, the UI observing the store rather than the network, a sync layer that reconciles, a conflict policy you can defend, pagination with stable cursors, and a clear answer for what the user sees when the network is gone.

Design a chat screen. Message ordering and identity, optimistic local sends with a pending state, retry and failure UI, pagination upward through history, and read receipts as a consistency problem.

Design analytics or a logging pipeline. Batching, disk-backed queueing so events survive a crash, flush on background, back-off on failure, and a hard rule about not logging personal data.

Tradeoffs, stated as tradeoffs

The single most common weakness in this round is presenting a design as though it had no cost. Every real choice has one, and naming it is the mark of experience.

- More aggressive caching means faster and cheaper, but staler.
- Offline-first means resilience, but you now own conflict resolution.
- Fewer, larger requests reduce overhead but delay the first pixel.
- A background sync improves perceived speed and spends battery.

> **What is actually being assessed**
>
> Not whether you produce the architecture the interviewer had in mind. They are checking whether you ask before assuming, structure a large problem into parts, know where the real complexity lives — caching, concurrency, failure — and can change your mind when given new information. A candidate who revises a decision after a hint is rated above one who defends a first idea to the end.

A rehearsal worth doing

Take one app on your phone and spend twenty minutes, out loud, designing its main screen: the states, the model, the caching, the threading, and what happens with no network. Do it three times with three different apps. It is the closest thing to the real round you can practise alone, and it is far more useful than reading another architecture article.

Part five

The pages you will actually come back to

Complexity tables, the Swift API you will reach for under pressure, and what to do with the forty-five minutes once the problem is on the screen.

Chapter forty-two

Complexity tables

One page to check yourself against. If you cannot state the complexity of your own solution, you are not finished with it.

Swift standard library

Operation	Array	Dictionary	Set	String
access by index	O(1)	—	—	O(n)
lookup by key	—	O(1) avg	O(1) avg	—
insert at end	O(1)*	O(1) avg	O(1) avg	O(1)*
insert at front	O(n)	—	—	O(n)
remove last	O(1)	—	—	O(1)
remove first	O(n)	—	—	O(n)
remove by key	O(n)	O(1) avg	O(1) avg	—
contains	O(n)	O(1) avg	O(1) avg	O(n·m)
count	O(1)	O(1)	O(1)	O(n)
sort	O(n log n)	—	—	—
min / max	O(n)	O(n)	O(n)	—

* amortised. Dictionary and Set costs are average-case; worst case is O(n) under heavy collisions.

Data structures you write yourself

Structure	Access	Search	Insert	Delete	Space
Stack	O(1) top	O(n)	O(1)	O(1)	O(n)
Queue, head index	O(1) front	O(n)	O(1)	O(1)*	O(n)
Linked list	O(n)	O(n)	O(1) at a node	O(1) at a node	O(n)
BST, balanced	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
BST, degenerate	O(n)	O(n)	O(n)	O(n)	O(n)
Heap	O(1) root	O(n)	O(log n)	O(log n)	O(n)
Trie	O(k)	O(k)	O(k)	O(k)	O(total chars)
Union-Find	—	O(α(n))	O(α(n))	—	O(n)

Patterns

Pattern	Typical time	Typical space
Two pointers	O(n), or O(n log n) with a sort	O(1)
Sliding window	O(n)	O(k)
Fast and slow pointers	O(n)	O(1)
Binary search	O(log n), or O(n log S) on the answer	O(1)
DFS	O(V + E)	O(depth)
BFS	O(V + E)	O(V)
Backtracking	$O(n \cdot 2^n)$ or $O(n \cdot n!)$	O(n)
Top-k with a heap	O(n log k)	O(k)
Dynamic programming	O(states × transitions)	O(states), often reducible
Intervals	O(n log n)	O(n)
Monotonic stack	O(n)	O(n)
Prefix sums	O(n)	O(n)
Union-Find	$O(E \cdot \alpha(n))$	O(V)

Reading the constraints

Constraint on n	What fits	Likely intended
$n \le 12$	O(n!)	permutations, brute force
$n \le 20$	$O(2^n)$	subsets, bitmask DP
$n \le 500$	$O(n^3)$	interval DP, Floyd–Warshall
$n \le 5{,}000$	$O(n^2)$	two-dimensional DP
$n \le 10^5$	O(n log n)	sorting, heap, binary search
$n \le 10^6$	O(n)	one pass, hash map
$n > 10^9$	O(log n) or O(1)	maths, or binary search on the answer

Chapter forty-three

Swift API quick reference

The calls you will want at speed, in one place. Everything here is standard library — no imports beyond Foundation, and most of it needs not even that.

Array

```
var a = [3, 1, 4]
Array(repeating: 0, count: n)
Array(0..<n)                        // [0, 1, ..., n-1]
Array(repeating: Array(repeating: 0, count: cols), count: rows)

a.count; a.isEmpty
a.first; a.last                     // Optional
a.append(5); a += [6, 7]
a.insert(0, at: 0)                  // O(n)
a.remove(at: 1); a.removeLast(); a.popLast()
a.removeAll(); a.reserveCapacity(100)

a.sort(); a.sorted()
a.sorted { $0 > $1 }
a.reverse(); a.reversed()
a.shuffled()
a.swapAt(0, 1)

a.min(); a.max(); a.reduce(0, +)
a.contains(3); a.firstIndex(of: 3)      // Optional
a.first { $0 > 2 }                      // Optional
a.allSatisfy { $0 > 0 }
a.map { $0 * 2 }; a.filter { $0 > 1 }
a.compactMap { Int($0) }                // drops nils
a.flatMap { $0 }                        // flattens one level
a.enumerated(); zip(a, b)
a.prefix(3); a.suffix(3); a.dropFirst(); a.dropLast()
a.split(whereSeparator: { $0 == 0 })
Array(Set(a))                           // deduplicate, order lost
```

Dictionary

```
var d: [String: Int] = [:]
d["k"] = 1
d["k"]                              // Optional
d["k", default: 0] += 1             // the counting idiom
d.removeValue(forKey: "k")
d.keys; d.values; d.count
d.updateValue(2, forKey: "k")       // returns the old value
d.merge(other) { current, _ in current }
d.sorted { $0.value > $1.value }    // returns an array of pairs
d.mapValues { $0 * 2 }
d.filter { $0.value > 1 }
for (k, v) in d { }                 // order undefined
```

Set

```
var s: Set<Int> = []
s.insert(1)                         // (inserted: Bool, memberAfterInsert: Int)
s.contains(1); s.remove(1)
s.union(t); s.intersection(t); s.subtracting(t); s.symmetricDifference(t)
s.isSubset(of: t); s.isSuperset(of: t); s.isDisjoint(with: t)
s.formUnion(t)                      // in place
```

String

```
var s = "hello world"
Array(s)                          // [Character] — do this once, then index
String(chars)                     // back again
s.count                           // O(n)
s.uppercased(); s.lowercased()
s.hasPrefix("he"); s.hasSuffix("ld"); s.contains("lo")
s.split(separator: " ")           // [Substring]
s.split(separator: " ").map(String.init)
s.replacingOccurrences(of: "l", with: "L")
s.trimmingCharacters(in: .whitespacesAndNewlines)
String(s.reversed())
String(s.sorted())                // canonical anagram key
s.firstIndex(of: "o")             // String.Index, not Int
String(repeating: "ab", count: 3)
```

Character

```
c.isLetter; c.isNumber; c.isUppercase; c.isLowercase
c.isWhitespace; c.isPunctuation
c.wholeNumberValue                // Optional<Int>
c.asciiValue                      // Optional<UInt8>
Character(UnicodeScalar(65 as UInt8))   // "A" — the UInt8 init is non-failable
Int(c.asciiValue! - Character("a").asciiValue!)
```

Numbers

```
Int.max; Int.min; Int.bitWidth
abs(-3); min(a, b); max(a, b)
a.quotientAndRemainder(dividingBy: b)
7 / 2                             // 3 — truncates
7 % 2                             // 1
(-7) % 2                          // -1 — sign follows the dividend
n.isMultiple(of: 2)
Double(n); Int(3.9)               // 3
Double(n).squareRoot()
pow(2.0, 10.0)                    // needs Foundation
n.nonzeroBitCount
Int.random(in: 1...6)
```

Negative modulo

Swift's `%` takes the sign of the left operand, so `-1 % 3` is `-1`, not `2`. When you need a genuine mathematical modulo — wrapping around a circular array — write `((a % n) + n) % n`.

Ranges and iteration

```
0..<n; 0...n
(0..<n).reversed()
stride(from: 0, to: n, by: 2)
stride(from: n - 1, through: 0, by: -1)
for (i, x) in a.enumerated() { }
for (x, y) in zip(a, b) { }
for x in a where x > 0 { }
```

Sorting with several keys

```
items.sorted {
    $0.priority == $1.priority ? $0.name < $1.name
                               : $0.priority > $1.priority
}
```

Chapter forty-four

The interview itself

You can know every pattern in this book and still interview badly. The last skill is running the forty-five minutes as a conversation rather than a performance.

The first five minutes are not wasted

The instinct under pressure is to start typing. Resist it. Restate the problem in your own words, confirm the input and output types, and ask about the cases that are not in the example:

- Can the input be empty? Can it be a single element?
- Are there duplicates? Negative numbers? Is it sorted?
- How large can it get? *(this is the complexity target, in disguise)*
- Is there always a valid answer, or can it fail?

Each of those is a question you would otherwise have discovered as a bug, in front of someone, later.

Think out loud, in the right register

Narrating every keystroke is noise. What is worth saying is the decisions: why this data structure, what you are trading away, what you have ruled out and on what grounds. "I could sort here, but that's O(n log n) and I think a set gets me O(n), so let me try that first" is one sentence that shows the entire reasoning process.

If you go quiet because you are stuck, say that too. "I'm going to think for a moment about how to handle duplicates" is a normal thing to say and infinitely better than thirty seconds of silence.

When you are stuck

> Work a small example by hand. Not the given one — a smaller one you make up. Most blocks break the moment you actually trace three elements on paper, because the pattern becomes visible in the trace.

If that fails, the moves in order are: state the brute force and get it working, then optimise; try the next simplest data structure and ask what it buys you; ask what information you keep recomputing, since that is almost always where the optimisation hides. And if you are genuinely stuck, ask for a hint. It costs far less than silence, and interviewers are usually glad to give one.

Write code you can defend

Name variables for what they hold — `left`, `right`, `seen`, `counts`, `best` — not `i`, `j`, `temp`, `x`. Keep functions short. Skip the clever one-liner if you would need thirty seconds to explain it. The person watching is assessing whether they would want to review your pull requests.

Test before you are asked

When the code is written, do not stop. Pick a small input and walk it line by line, out loud, updating the variables as you go. This catches off-by-one errors, and it demonstrates a habit that matters more than the answer.

Then check the edges deliberately: empty input, one element, all-identical elements, the largest allowed value. Say what each one does.

Finish properly

State the final time and space complexity. Then add one honest sentence about what you would change with more time — a cleaner data structure, a way to halve the memory, a test you would write. Ending on a forward-looking note leaves a much stronger impression than trailing off when the code compiles.

The last five minutes are yours

You will be asked if you have questions. Have two. Ask about something real — how the team decides what to build, what the code review culture is like, what the hardest part of the codebase is. It is the one part of the interview where you are assessing them, and treating it as such reads as confidence.

What is actually being measured

Very few interviewers care whether you produce the optimal solution unaided. They are answering one question: what would it be like to solve a problem with this person on a Tuesday afternoon. Clear thinking, honest uncertainty, and receptiveness to a hint all count towards that. Silent brilliance does not.

A four-week drill, if you want a structure

Week 1 — part one and part two, writing every data structure from memory once. **Week 2** — chapters 19 to 25, four problems each, template first from memory. **Week 3** — chapters 26 to 32, same method. **Week 4** — mixed problems with no idea which pattern applies, timed at forty minutes, out loud. That last week is the one that transfers, and it is the one people skip.

Run part four in parallel rather than after: one chapter an evening, and for each one, find the equivalent in a codebase you have written. The iOS rounds are not something you cram the night before, because the questions are about experience rather than recall.